



Budapest University of Technology and Economics
Faculty of Electrical Engineering and Informatics
Department of Telecommunication and Media Informatics

Praneshraj Tiruppur Nagarajan Dhyaneswar

ENERGY SAVING OPERATIONS OF 5G AND 6G NETWORKS USING AI

SUPERVISOR

Dr. Tibor Cinkler

BUDAPEST, 2026

Abstract.....	5
Összefoglaló.....	6
1. Introduction.....	7
1.1. Motivation.....	7
1.2. Problem Statement.....	8
1.3. Contributions and Thesis Organisation.....	8
2. Network Topology and Configuration.....	9
2.1. Simulation Environment and Hardware.....	9
2.2. Four-Tier HetNet Deployment.....	9
2.3. MIMO and Beamforming Model.....	10
3. Radio Propagation Models.....	11
3.1. Path-Loss Model.....	11
3.2. Received Power.....	12
3.3. SINR and Feasibility.....	12
3.4. Load-Dependent Power Consumption Model.....	14
4. Exact Problem Formulation.....	16
4.1. Integer Linear Program.....	16
4.2. Set Cover Reduction.....	17
4.3. Diurnal Traffic Model.....	17
4.4. Greedy Capacitated Set-Cover Heuristic.....	18
5. Topology-Resource Utility Score (TRUS).....	20
5.1. Mitigating Circular Dependency in Load Estimation.....	20
5.2. Definition of TRUS.....	20
5.3. Channel Utility, Criticality, and Relevance Metrics.....	22
5.4. TRUS Integration into the Graph Neural Network.....	23
6. Preliminary Methodological Investigation.....	24
6.1. Supervised Learning.....	24
6.2. Temporal GNN.....	24
6.3. Exhaustive Search.....	24
6.4. Challenges Identified by the Screening.....	25
7. Semi-Supervised Graph-SCP for HetNet Base Station Activation.....	25
7.1. Design Rationale.....	26
7.2. Dataset Composition.....	27
7.3. GNN Construction.....	29
7.4. GraphSAGE Architecture.....	30
7.5. Semi-Supervised Loss Function.....	31
7.6. Graph-SCP Pipeline.....	32
7.7. Training and Convergence.....	32
7.8. Evaluation: Power, Coverage and Optimality Gap.....	34
7.9. Instance-Level Deep Dive.....	37
7.9.1. Best Case: Graph-SCP Matches the Oracle.....	37
7.9.2. Worst Case: Graph-SCP Failure Analysis.....	39
8. Ablation Study of TRUS within Graph-SCP.....	42
8.1. Ablation Design.....	42
8.2. Ablation Results.....	42

8.3. Remaining Challenges of Graph-SCP	44
9. Reinforcement Learning as Baseline: PPO for HetNet.....	45
9.1. Markov Decision Process Formulation.....	45
9.2. Proximal Policy Optimisation and Total Loss.....	47
9.3. Mobility and Handover Modelling.....	48
9.4. Policy Architecture and Hyperparameters.....	49
9.5. Training Behaviour.....	50
9.6. Policy Evaluation and Benchmarking.....	52
9.7. Handover Evaluation and Spatial Analysis.....	54
10. Limitations and Future Work.....	57
10.1. Optimality and Decision Time in Graph-SCP.....	57
10.2. Reinforcement Learning and Coverage Safety.....	57
10.3. Simulation Fidelity and Channel Modelling.....	58
Summary.....	59
References.....	60
Appendix.....	63
A. Algorithmic Implementations.....	63
A.1. Greedy Capacitated Set Cover Heuristic Algorithm.....	63
A.2. TRUS Feature Integration.....	63
B. Source Code Availability.....	64
B.1. Graph-SCP Implementation.....	64
B.2. Reinforcement Learning (RL-PPO) Implementation.....	64
C. Hyperparameter Configurations.....	65
C.1. Graph-SCP Training Configuration.....	65
C.2. RL-PPO Training Configurations.....	65
Acknowledgements.....	66

STUDENT DECLARATION

I, **Praneshraj Tiruppur Nagarajan Dhyaneswar**, the undersigned, hereby declare that the present MSc thesis work has been prepared by myself and without any unauthorized help or assistance. Only the specified sources (references, tools, etc.) were used. All parts taken from other sources word by word, or after rephrasing but with identical meaning, were unambiguously identified with explicit reference to the sources utilized.

I authorize the Faculty of Electrical Engineering and Informatics of the Budapest University of Technology and Economics to publish the principal data of the thesis work (author's name, title, abstracts in English and in a second language, year of preparation, supervisor's name, etc.) in a searchable, public, electronic and online database and to publish the full text of the thesis work on the internal network of the university (this may include access by authenticated outside users). I declare that the submitted hardcopy of the thesis work and its electronic version are identical.

Full text of thesis works classified upon the decision of the Dean will be published after a period of three years.

Budapest, 08 June 2026

.....
Praneshraj Tiruppur Nagarajan Dhyaneswar

Abstract

Base stations (BS) command the energy budget of a radio access network, with most of this energy being consumed as fixed overhead that does not scale down with traffic unless the equipment is intentionally put to sleep. As the fifth generation of cellular mobile communications (5G) densifies toward 6G with the deployment of macro, micro, pico, and femto cells and higher frequency carriers, keeping all of the base stations active during off-peak hours is uneconomical. This thesis develops and evaluates a data-driven framework that selects the minimum-power set of BSs required to effectively cover the served user-equipment (UE) for any given traffic snapshot, modelling the activation decision as a capacitated, power-weighted set cover problem to identify the most energy-efficient subset of base stations that maintains acceptable signal quality for all subscribers.

A custom four-tier heterogeneous network simulator was developed to model multi-band radio conditions, load-dependent power consumption, and realistic traffic patterns, providing the foundation to benchmark a graph neural network (GNN) against an exact optimization baseline. A novel contribution is the Topology-Resource Utility Score (TRUS), a metric that assesses a base station's importance by combining signal quality with an estimate of available capacity. The GNN is trained using a semi-supervised approach, balancing a small set of optimal solutions with an unsupervised loss function that directly penalizes energy costs and coverage failures. Finally, a reinforcement learning (RL) agent is developed independently to incorporate user mobility and handover events, determining whether a learned policy can shift base station management from reactive adjustments to proactive, energy-efficient operations.

Keywords: Base stations, data-driven, user equipment, set cover problem, heterogeneous network, HetNet, MIMO, graph neural network, GNN, optimization, Topology-Resource Utility Score, TRUS, handover, semi-supervised, reinforcement learning, agent, policy

Összefoglaló

A bázisállomások (BS-ek) uralják a rádiós hozzáférési hálózat energiafelhasználását, amelynek jelentős része olyan fix energiaigényből származik, amely a forgalom csökkenésével nem mérséklődik, kivéve, ha a berendezéseket szándékosan alvó üzemmódba helyezik. Ahogy az ötödik generációs mobilkommunikációs rendszerek (5G) a makro-, mikro-, piko- és femtocellák, valamint a magasabb frekvenciasávok alkalmazásával a 6G irányába fejlődnek és sűrűsödnek, gazdaságtalanná válik az összes bázisállomás folyamatos üzemeltetése az alacsony forgalmú időszakokban. Jelen diplomamunka egy adatvezérelt keretrendszert fejleszt és értékel, amely minden adott forgalmi pillanatkép esetén kiválasztja a felhasználói berendezések (UE-k) hatékony kiszolgálásához szükséges minimális energiaigényű bázisállomás-készletet. Az aktiválási döntést kapacitáskorlátos, teljesítménysúlyozott halmazfedési problémaként modellezi annak érdekében, hogy meghatározza a legergiahatékonyabb bázisállomás-részhalmazt, amely minden előfizető számára megfelelő jelminőséget biztosít.

Egy egyedi, négy szintű heterogén hálózati szimulátor készült a többfrekvenciás rádiós környezet, a terhelésfüggő energiafogyasztás és a valósághű forgalmi mintázatok modellezésére, amely alapot biztosított egy gráf neurális hálózat (GNN) és egy egzakt optimalizációs referenciaeljárás összehasonlító értékeléséhez. A munka egyik újszerű hozzájárulása a Topology-Resource Utility Score (TRUS) mutató, amely a bázisállomások fontosságát a jelminőség és a rendelkezésre álló kapacitás becslésének kombinálásával értékeli. A GNN tanítása félig felügyelt tanulási megközelítéssel történik, amely kis számú optimális megoldást egy olyan felügyelet nélküli veszteségfüggvénnyel egyesít, amely közvetlenül bünteti az energiafelhasználást és a lefedettség hiányát. Végül egy megerősítési tanulási (Reinforcement Learning, RL) ügynök kerül önállóan kidolgozásra a felhasználói mobilitás és a handover események figyelembevételére annak vizsgálatára, hogy egy tanult stratégia képes-e a bázisállomások kezelését a reaktív beavatkozások helyett proaktív, energiahatékony működés irányába elmozdítani.

1. Introduction

1.1. Motivation

Even when idle, a powered radio equipment from a base station draws substantial power. Unless the equipment is actively put into a low-power or sleep state, measuring campaigns show that much of this is fixed overhead. The problem gets worse as networks transition from 5G to 6G. Delivering higher capacity requires deploying more cells and using high-frequency carriers, also known millimeter wave (mmWave) frequency bands [19]. That forces denser small cell deployment. Mobile traffic follows a strong diurnal pattern, meaning the network stays over-provisioned for most of the day. This creates an opportunity to save energy by turning off redundant base stations during off-peak hours and waking them up when demand returns [1, 9].

To model this complexity, the thesis looks at a multi-tier heterogeneous network (HetNet). It's a layered setup where each type of base station plays a role. Macro cells offer dependable, broad coverage, but they need a significant amount of electricity to run. For a fraction of the cost, smaller micro, pico, and femto cells boost localized capacity. The best way to cut energy use is to intelligently manage the sleep cycles of these tiny cells, since they're much cheaper to operate. The main challenge is finding the most energy-efficient mix of these tiers while still keeping every active user connected, and this puzzle keeps changing as user density and the spatial arrangement of user positions shift.

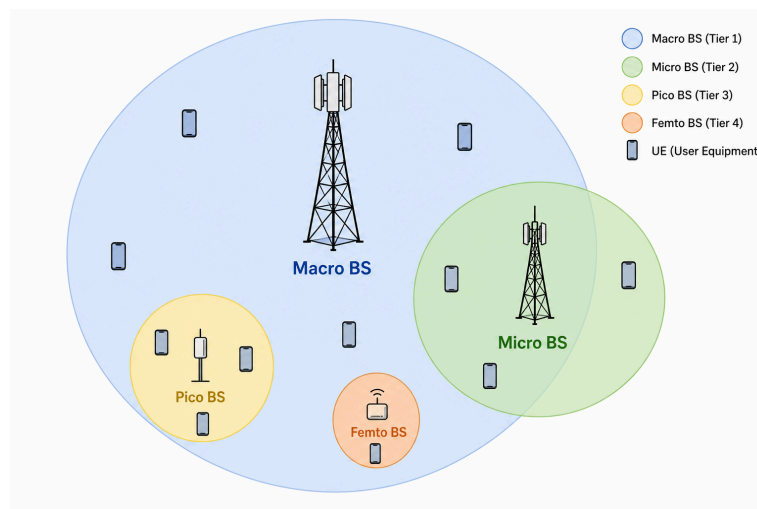


Figure 1.1: Four-tier HetNet topology with macro, micro, pico, and femto BSs

1.2. Problem Statement

Naively turning off base stations degrades coverage. At each traffic snapshot, a practical controller must solve a complex combinatorial problem, to find a binary activation vector that minimizes total network power while respecting capacity and signal quality constraints. Framed as a capacitated, power-weighted set-cover problem, this task is NP-hard due to per-BS user capacities, interference coupling, and heterogeneous operating costs across tiers. This complexity mirrors Fayad and Cinkler's proof that joint user and power allocation is NP-hard via reduction to the Generalized Assignment Problem [2, 22]. These interactions make the optimal configuration highly sensitive to small shifts in user distribution. While Integer Linear Programming (ILP) solves this optimally, it is too slow for real time control. This tension between latency and optimality motivates a learned approximation. The inherent relational representation driven by distance, competition, and interference between base stations and users maps naturally to a graph [21]. A graph neural network (GNN) learns node representations by repeatedly aggregating information from each node's neighbours through message passing. After several rounds of execution, a prediction head maps these final node representations to a target output. The inductive GraphSAGE (sample and aggregate) [3] variant samples and aggregates neighbour features so that the same learned operator transfers to graphs of different sizes. Consequently, GraphSAGE, which easily adapts to varying user counts, provides a robust framework to approximate the optimal solution in a fraction of the time.

1.3. Contributions and Thesis Organisation

This thesis proposes a semi-supervised graph neural network, trained on exact ILP labels for optimizing base station activation [15]. A core contribution is TRUS (Topology-Resource Utility Score), a pre-association feature that fuses channel quality with an expected load-slack estimate to avoid label leakage and circular dependencies. This static framework is benchmarked against an exact solver for power savings, coverage, and latency. The work then extends into dynamic control with an exploratory reinforcement learning agent that models user mobility and handover mechanics. The remainder reviews related literature including the Graph-SCP (G-SCP) framework, the system model, the core GNN framework and TRUS metric, the dynamic RL extension, experimental evaluations, and a final conclusion.

2. Network Topology and Configuration

2.1. Simulation Environment and Hardware

The simulation environment was developed in Python 3.10 and hosted on Google Colab. The setup utilized a virtual instance equipped with 12 GB of system RAM. The workload was intentionally partitioned across both the CPU and GPU. The environment handled network physical computation, greedy baseline execution, and exact Integer Linear Programming (ILP) solving via the PuLP [4] /CBC framework. Meanwhile, an NVIDIA T4 GPU was exclusively used to accelerate tensor operations, PyTorch processes, and the graph message-passing forward passes for the GraphSAGE model.

2.2. Four-Tier HetNet Deployment

The main topology places **21 base stations** within an **800 m × 800 m** area. The simulator builds a four-tier Heterogeneous Network (HetNet) with macro, micro, pico, and femto base stations. All experiments use a primary reference scenario with 21 base stations (3 macro, 6 micro, 8 pico, and 4 femto) deployed in this area. Base station locations are generated using a *maxi-min* placement algorithm, which maximizes the minimum distance between any two sites. Upon calculation, this algorithm enforces tier-specific separation constraints, ensuring that macro, micro, pico, and femto base stations maintain minimum inter-site distances of 400 m, 150 m, 80 m, and 30 m, respectively. The maximum capacities of 200, 80, 30 and 10 users for macro, micro, pico and femto cells respectively were empirically chosen to reflect standard telecommunication planning assumptions for dense urban 5G network densification. [5]

Tier	Band	Number of Transmitter Antennas	Base Station Gain (dBi)	Radius (m)	Maximum User Capacity
Macro	700 MHz	8	8	2000	200
Micro	3.5 GHz	64	8	500	80
Pico	28 GHz	256	5	200	30
Femto	28GHz	256	5	50	10

Table 2.1: Per-tier physical and operational configurations for the simulated heterogeneous network

At the receiver, each user equipment (UE) utilizes dual antennas ($N_{RX} = 2$) with 0 dBi

gain, while base station coverage radii are enforced as hard geometric clips within the path-loss model. The frequency allocation reflects established physical deployment strategies. The 700 MHz macro tier (wavelength $\lambda \approx 43$ cm) diffracts strongly around obstacles, providing a reliable 2 km coverage floor. The 3.5 GHz micro tier ($\lambda \approx 8.6$ cm) offers an optimal balance of reach and capacity at 500 m. Finally, the 28 GHz pico and femto tiers ($\lambda \approx 1.07$ cm) deliver massive hotspot capacity and localized, low-power reuse. Because 28 GHz signals suffer roughly 32 dB higher free-space loss than 700 MHz at equivalent distances, their transmission ranges are strictly hard-clipped to 200 m and 50 m, respectively.

2.3. MIMO and Beamforming Model

In a base station, Multiple-Input Multiple-Output (MIMO) technology means using multiple antennas to transmit and receive signals so the BS can improve beamforming gain, link reliability, and spectral efficiency. In the simulation, MIMO is incorporated into the system at the link-budget level via array gain, rather than as a complex spatial-multiplexing channel model. The beamforming gain provided by a base station is calculated as:

$$G_j^{bf} = 10 \log_{10}(N_{TX,b}), \quad \text{where } N_{TX} \in \{8, 64, 256\} \quad (3.1)$$

G_j^{bf} denotes the beamforming gain provided by the j^{th} base station in logarithmic scale (dB), which is directly determined by the number of transmitting antennas N_{TX} actively deployed in frequency band b . This formulation yields beamforming gains of 9 dB for macro cells, 18 dB for micro cells, and a substantial 24 dB for both pico and femto cells. This 24 dB gain is the critical mechanism that makes the 28 GHz small-cell sleep strategy energetically viable. Without it, the severe path loss of the high-frequency mmWave band would render these small cells geometrically unusable, effectively eliminating any energy-saving potential from dense small-cell sleeping. Please note that simplifying MIMO to just this array-gain view, instead of simulating full spatial multiplexing, is a deliberate limitation which will be discussed in [Chapter 10](#).

3. Radio Propagation Models

3.1. Path-Loss Model

Based on the 3GPP TR 38.901 [7] technical report, I implement the dual-slope, distance and frequency-dependent path loss where they are available in the 9.5 - 100 GHz range for urban macro (UMa) and urban micro (UMi) deployments. As we do not include buildings and objects in our simulation, the line-of-sight (LoS) case was selected.

Given a UE-BS pair (i, j) with carrier frequency f_c and separated by d meters in band index b , the path loss is defined as follows:

For the macro 700 MHz band ($b = 0$):

$$PL_1(d, f_c) = 28 + 22 \log_{10}(d) + 20 \log_{10}(f_c) \quad (3.2)$$

$$PL_2(d, f_c) = 28 + 40 \log_{10}(d) + 20 \log_{10}(f_c) - 9 \log_{10}(d_{BP}^2) \quad (3.3)$$

For the mid and high band tiers ($b = 1, 2$):

$$PL_1(d, f_c) = 32.4 + 21 \log_{10}(d) + 20 \log_{10}(f_c) \quad (3.4)$$

$$PL_2(d, f_c) = 32.4 + 40 \log_{10}(d) + 20 \log_{10}(f_c) - 9.5 \log_{10}(d_{BP}^2) \quad (3.5)$$

Here, the breakpoint distance d_{BP} is calculated as:

$$d_{BP} = \frac{4(h_{BS}-1)(h_{UE}-1)f_c}{3 \times 10^8} \quad (3.6)$$

Note that base station antenna height h_{BS} is set to 25 m for macro cells and 10m for small cells. UE height is fixed at 1.5 m. We apply PL_1 if the distance d is less than or equal to d_{BP} , otherwise PL_2 . To account for real-world obstacles and fading, we add log-normal shadowing to the PL per tier with standard deviations: macro 4 dB, micro 6 dB, pico 8 dB, femto 10 dB.

3.2. Received Power

The received power at UE i from base station j in dBm is given by:

$$P_{ij}^{rx}(dBm) = P_j^{tx} + G_j^{BS} + G^{UE} + G_j^{bf} - PL_{ij} \quad (3.7)$$

where G_j^{BS} is the per-tier antenna gain (8,8,5,5 dBi respectively for macro, micro, pico, femto), $G^{UE} = 0$ dBi, P_j^{tx} is the maximum physical RF output power radiated by the antennas at full capacity and G_j^{bf} is defined in equation [3.1](#).

3.3. SINR and Feasibility

Once we convert the received power Eq. [\(3.7\)](#) to a linear scale, we can calculate the Signal-to-Interference-plus-Noise-Ratio (SINR) [[14](#)] at UE i from BS j as:

$$SINR_{ij} = \frac{P_{ij}^{rx}}{\sum_{k \neq j} a_k P_{ik}^{rx} + N_i} \quad (3.8)$$

$a_k \in \{0, 1\}$. Its value is 1 if BS k is active (i.e. interferer).

N_i is the thermal noise power at UE _{i} . Note that co-channel interference is summed over same-band active BSs only.

Thermal noise is derived from the standard spectral noise density formula:

$$N_0(dBm) = -174 + 10 \log_{10}(B_{Hz}) + NF_{UE} \quad (3.8)$$

For the bandwidths (B), values of 20 MHz, 60 MHz, and 400 MHz were used for the 700 MHz, 3.5 GHz, and 28 GHz bands respectively, alongside an assumed standard UE noise figure (NF_{UE}) of 9 dB. Because the UE has two receive chains ($N_{RX}=2$), the total effective noise is expressed as

$$N_i = N_0 \times N_{RX} \quad (3.9)$$

To determine if a connection is viable, I defined a binary feasibility indicator, F_{ij} . For a BS j to feasibly serve a UE i (i.e. $F_{ij}=1$), two physical conditions have to be met:

$$F_{ij} = 1 [d_{ij} \leq R_j \wedge SINR_{ij}(dB) \geq \Gamma] \quad (3.10)$$

$$\Gamma = -6 \text{ dB} \quad (3.11)$$

Empirically, I chose a permissive SINR threshold of $\Gamma = -6 \text{ dB}$ to accommodate cell-edge UEs, i.e. users located at the very boundary of a BS's coverage where the signal is weakest and interference is highest. Including these edge users deliberately stresses the interference coupling between active cells, creating a much more realistic optimization challenge than a strict threshold would.

The load ratio ρ for each BS j is the ratio of the sum of association y (i.e. user i connected to BS j) over the maximum capacity of that BS:

$$\rho_j = \frac{\sum_i y_{ij}}{C_j}, \quad 0 \leq \rho_j \leq 1 \quad (3.12)$$

Here, C_j represents the maximum user capacity for macro, micro, pico, and femto cells (200, 80, 30, 10 respectively). This capacity constraint forces the optimizer to choose between keeping a power-hungry macro cell on or turning it off and running several smaller cells instead. The interference coupling in Eq. 3.8 is what makes this optimization problem challenging. Without interference, a UE's connection depends only on its direct signal, making the problem easy to solve by simple greedy algorithms. But activating a base station adds noise to every UE within range, potentially dropping a connected UE below the Γ threshold. As a consequence, the covering sets are not static, they shift depending on the rest of the active network. Through empirical calibration, the hardware SINR was set to 40 dB, adhering to the upper limits implied by 3GPP TS 38.214 [13].

3.4. Load-Dependent Power Consumption Model

Each base station follows a load-dependent linear power model from the EARTH D2.3 and Auer et al. framework as follows [6, 8]:

$$P_j(\rho_j) = P_j^0 + \Delta_{p,j} \cdot P_j^{tx} \cdot \rho_j \quad \text{if BS } j \text{ is active} \quad (3.12)$$

$$P_j^{sleep} \quad \text{if BS } j \text{ is inactive}$$

- $x_j \in \{0, 1\}$ is the binary decision variable representing activation state of BS j
- P_j^0 is the fixed baseline power consumed by an active BS at zero data load
- $\Delta_{p,j}$ is the load-dependent slope which is a scaling factor reflecting the inefficiency of the power amplifier
- P_j^{tx} is the maximum physical RF output power radiated by the antennas at full capacity
- P_j^{sleep} is the sleep-mode power consumption of BS j

And the total power of all BSs is given by:

$$P_{total} = \sum_j [x_j P_j(\rho_j) + (1 - x_j) P_j^{sleep}] \quad (3.13)$$

Tier	P ⁰ (W)	Δ _p (W)	P ^{tx} (W)	P ^{sleep} (W)	P ^{max} (W) (100% load)	Deactivation savings (W)
Macro	354.0	4.7	40.0	17.7	542.0	336.3
Micro	56.0	2.6	6.3	2.8	72.4	53.2
Pico	6.8	4.0	0.25	0.34	7.8	6.5
Femto	4.8	8.0	0.10	0.24	5.6	4.6

Table 3.1: Base station power consumption parameters per tier. Columns 2,3,4 and 5 are based on the EARTH linear model [6], highlighting the dominance of fixed idle power (P⁰) and deactivation savings

If all BSs in our simulation operate at maximum capacity, the network draws **2145.1 W**. Also, the table shows that idle power P^0 dominates this energy budget. For example, two-thirds of a macro cell's peak power (354 W out of 542 W) is just the fixed cost of staying awake, while a fully loaded data transmission only adds 188 W. Femtocells follow the same trend, with an idle power (4.8 W) that completely dwarfs the load-dependent slope (0.8 W).

This **heavy baseline cost** justifies the key design choice in this thesis. Because simply balancing traffic between active cells saves very little energy, the optimization must be formulated as a binary set cover problem focused on switching stations off. The deactivation saving column shows that sleeping a macro recovers a massive 336.3 W. This is the controller's most valuable yet riskiest move, feasible only when the underlying small cells can fully guarantee coverage.

4. Exact Problem Formulation

4.1. Integer Linear Program

In order to find the mathematically optimal base station configuration, I formulated the problem as an Integer Linear Program (ILP) [23]. Let $x_j \in \{0,1\}$ indicate if the base station is active, $y_{ij} \in \{0,1\}$ indicate if UE i is associated with BS j , and $z_i \in \{0,1\}$ indicate that UE i is successfully covered. The objective is to minimize the total network power (Eq. 4.1) subject to the constraints presented after it.

$$\min_{x,y,z} \sum_{j=1}^{21} [x_j(P_j^O + \Delta_{p,j} P_j^{tx} \rho_j) + (1 - x_j)P_j^{sleep}] \quad (4.1)$$

1. $y_{ij} \leq F_{ij}$ (*Feasible associations*): The UE-BS associations are only allowed on links that are feasible.
2. $y_{ij} \leq x_j$ (*Active BS requirement*): A UE can be associated with a BS only if the BS is active.
3. $z_i \leq \sum_{j=1}^{21} y_{ij}$ (*Coverage decoding*): A UE is marked as covered if it is associated with at least one base station.
4. $\sum_i y_{ij} \leq C_j x_j$ (*Capacity constraint*): The count of UEs served by base station j cannot exceed its capacity.
5. $\sum_i z_i \geq \tau U$ (*Minimum Coverage*): The total count of covered UEs must be at least a target fraction τ of the total number of UEs U .
6. $x_j, y_{ij}, z_i \in \{0, 1\}$ (*Integrality*): All variables are binary

The inherently linear load ratio ρ_j mathematically depends on the binary association variables y , I linearized the load term over the integer association counts. Since all decision variables (x,y,z) are constrained to binary integers, the problem operates as a strict ILP solvable by branch-and-cut algorithms (using PuLP + CBC) [4]. The ILP is solved in full mode, i.e. coverage z and associations y are decoded exclusively from the mathematical constraints without any greedy fallback, ensuring the optimum is completely faithful to the physical model.

Sometimes 100% network coverage just isn't physically possible. For those cases, I built a coverage-threshold fallback sequence. The solver first tries full coverage at $\tau = 1$. If that fails, it gradually lowers the target through a set of steps [1.0, 0.98, 0.96, 0.94, 0.92, 0.90, 0.85, 0.80, 0.75, 0.70, 0.65, 0.60] until a feasible solution turns up. During dataset generation, a strict 60-second ILP time limit per instance is enforced.

4.2. Set Cover Reduction

The ILP formulation effectively reduces our physical network model to a capacitated, weighted Set Cover Problem (SCP). We can attribute the universe of UEs as the total user population U , and the 'covering' set for each base station j as the specific UEs it can feasibly serve:

$$S_j = \{i \in U: F_{ij} = 1\} \quad (4.2)$$

Each set incurs a power cost $c_j \approx P_j$ and the objective is to select a minimum-cost subset of active BSs to cover the target fraction τ of users, while respecting each BS's hardware capacity limit:

$$\sum_i y_{ij} \leq C_j \quad (4.3)$$

The standard Set Cover problem is NP-hard, and the thesis' capacitated and weighted variant is computationally prohibitive for large networks [20]. This combinatorial complexity is the sole reason why relying on an exact ILP solver does not scale for fast, real-time network decisions. This hard mathematical limitation justifies my transition toward the scalable, learned approximation (Graph-SCP) developed in Chapter 7.

4.3. Diurnal Traffic Model

To evaluate the network under realistic conditions, I modeled the offered traffic using a 12-hour diurnal profile spanning from 10:00 to 22:00. The ~ 2.1 times dynamic range is a deliberate design choice to provide the necessary variance in the network load to make energy-saving sleep strategies viable and rigorously testable.

The UEs are deployed spatially using two distinct models:

1. Uniform: The baseline scenario where users are scattered randomly across the geographic area.
2. Hotspot: The clustered scenario where users are heavily concentrated into 5 to 8 Gaussian focal points (with a standard deviation of $\sigma = 35$ m), mimicking high density urban environments like city centres or transport hubs.

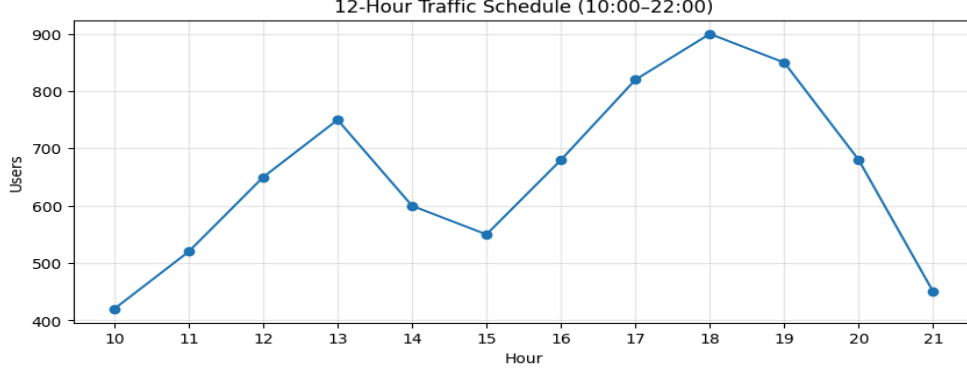


Figure 4.1: Twelve-hour diurnal UE traffic profile used in this thesis (10:00–22:00)

In the figure, The peak of 900 UEs at 18:00 and trough of 420 UEs at 10:00 span the dynamic range over which energy savings from selective BS deactivation become worthwhile. Please note that the final evaluation phase will deliberately test the trained model against Out-of-Distribution (OOD) user counts (e.g., beyond 900 UEs) to assess its generalization capability under extreme network stress.

4.4. Greedy Capacitated Set-Cover Heuristic

To provide a faster and interpretable reference for evaluating the proposed model, I implemented a standard greedy heuristic. At each iteration, the algorithm evaluates all currently inactive base stations by computing a cost-effectiveness ratio. This ratio is defined as the number of newly covered, previously unconnected UEs divided by the incremental power required to activate the station. The algorithm selects the base station with the highest score, activates it subject to its hardware capacity limit, and assigns the corresponding UEs. The scores of the remaining inactive stations are then dynamically recalculated. This iterative process continues until the network achieves the macroscopic coverage target τ , or until no feasible candidate stations remain. The complete, step-by-step pseudocode for this greedy baseline is provided in [Appendix A](#).

The greedy baseline is computationally efficient and reliably generates a physically

feasible solution. But it is fundamentally sub-optimal regarding energy conservation. Its approach is myopic, making locally optimal choices without ever reconsidering past activations, and obvious failure to execute the complex, global trade-offs that the ILP objective inherently rewards. For example, the greedy algorithm struggles to recognize scenarios where deactivating a single high-power macro cell is energetically superior to waking up multiple overlapping small cells, simply because the macro cell appeared highly cost-effective during an early iteration.

5. Topology-Resource Utility Score (TRUS)

5.1. Mitigating Circular Dependency in Load Estimation

A highly effective learned activation model requires input features that capture the most important characteristics of each base station:

1. The quality of its coverage (Channel State)
2. Expected traffic load

Coverage quality can be derived directly from geometry and propagation conditions, since it reflects how well a base station can serve the surrounding users. In contrast, traffic load is more difficult to estimate. The realized load ρ_j depends on the final user association decisions, which are exactly what the model is designed to predict. Using realized load as an input feature during training introduces a direct circular dependency, essentially causing label leakage. The feature is available during the training phase because the ILP optimizer has provided the ground-truth labels. But it remains fundamentally unknown at inference time prior to association. To resolve this causal paradox, we introduce the Topology-Resource Utility Score (TRUS), to estimate the expected load strictly from network geometry prior to any association decisions.

5.2. Definition of TRUS

The TRUS for a given base station is defined as:

$$TRUS_j = \alpha \cdot \Phi_{c,j} + \gamma_t \cdot (1 - \rho_j) \quad (5.1)$$

$\Phi_{c,j}$ represents a normalized channel quality metric

ρ_j denotes the expected normalised load

$(1 - \rho_j)$ captures the available capacity (slack)

α and γ_t are tunable weighting hyperparameters

Intuitively, a base station with high channel quality and substantial spare capacity receives a high TRUS score. Conversely, a base station with weak channel conditions or expected to be heavily saturated is penalized. In our semi-supervised Graph-SCP learning framework, value $\alpha = 0.5$ was set empirically. Similarly, in the reinforcement learning environment, the parameters are tuned to $\alpha = 0.5$ and $\gamma_t = 0.3$.

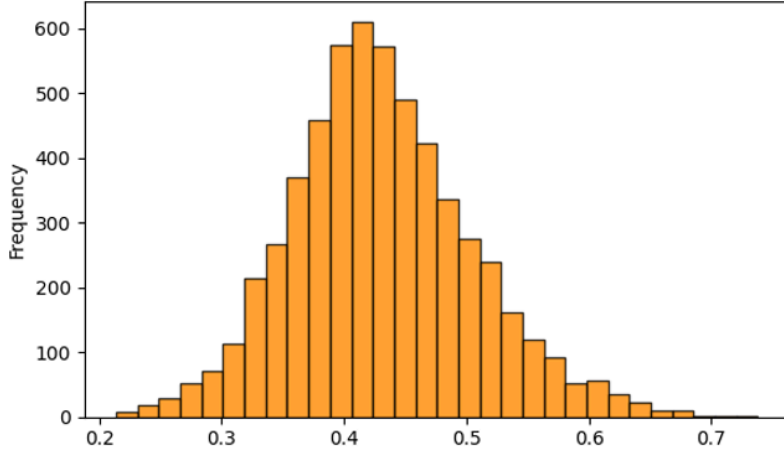


Figure 5.1: Empirical distribution of the mean TRUS across 5,680 ILP simulated network instances

The histogram demonstrates a near-normal distribution centered around a mean of 0.43, confirming that the pre-association TRUS metric carries discriminative information and avoids degeneracy across diverse base station topologies

To compute the expected load $\hat{\rho}_j$ without invoking the solver's association variables y , we provisionally assign each UE to its most geometrically plausible base station. This is done via a tier-weighted Voronoi-style partition of the service area based on the maximum feasible channel gain. The UEs are counted per base station and normalized by the base station's capacity C_j :

$$\bar{n}_j = |\{i: j = \arg \min_k w_k \cdot d_{ik}, F_{ij} = 1\}| \quad (5.2)$$

$$\hat{\rho}_j = \min(1, \frac{\bar{n}_j}{C_j}) \quad (5.3)$$

$\bar{n}_j$ is the provisional count of UEs assigned to BS j

w_k is tier-specific weight multiplier for base station k

d_{ik} is the Euclidean distance between UE i and BS k

F_{ij} is the feasibility indicator

C_j is the capacity of BS j

Since this provisional assignment relies exclusively on node positions, tier reach, and radio geometry, it is identically computable at both training and inference times, thereby eliminating label leakage. Even though this load estimate is only an approximate guess, it still helps steer the model toward base stations that are more likely to have free capacity. If this estimate is not accurate in some cases, the graph neural network can learn to rely on it less.

5.3. Channel Utility, Criticality, and Relevance Metrics

The coverage utility $\Phi_{c,j}$ aggregates the capacity of base station j to serve proximate UEs over its feasible set:

$$\Phi_{c,j} = \frac{1}{Z} \sum_{i:F_{ij}=1} g(\text{SINR}_{ij}) \quad (5.4)$$

To complement TRUS, two global structural metrics are integrated into the node-feature vectors to provide topological awareness into graph G that local message passing struggles to efficiently capture:

1. *Connectivity Estimate* (CE): It evaluates a BS's criticality to network connectivity. It approximates the drop in the algebraic connectivity (Fiedler value λ_2) of the graph Laplacian if node j were removed, utilizing a first-order approximation from the Fiedler eigenvector u_2 :

$$CE_j \approx \lambda_2(G) - \lambda_2(G \setminus j) \quad (5.5)$$

2. *Random-Walk-with-Restart* (RWR): It propagates importance through the bipartite UE-BS incidence structure, identifying base stations critical to covering the broader UE universe:

$$r = (1 - c)Wr + ce, \quad \text{RWR}_j = r_j \quad (5.6)$$

where W is the column-normalized adjacency matrix, e is the restart distribution, and c is the restart probability.

5.4. TRUS Integration into the Graph Neural Network

In [Chapter 7](#), to make the Graph-SCP architecture fully aware of network load, we implement the TRUS score into both the nodes and the edges of the graph:

1. As a node feature, TRUS is incorporated into 10-D base station feature vector. It sits alongside structural importance metrics like $\Phi_{c,j}$
2. As an edge feature, every link between a UE and a BS carries a 2D feature vector

By embedding TRUS directly onto the edges, the graph convolution layers can dynamically control how much a base station influences a UE's hidden representation. If a base station has high signal quality and plenty of spare capacity, its messages are given a stronger mathematical weight during the learning process.

The core purpose by framing TRUS is to prevent the model from cheating by looking into the future. Actual network load is the result of how users connect, so if we feed that actual load into the model, we are essentially giving it the answer key. To fix this chicken-and-egg problem, we use the Voronoi load, which only relies on fixed starting conditions like the map of where users and towers are.

By giving the model an early causally-upstream estimate, it learns to expect congestion without being handed the final solution. So, if a desired feature overlaps with the target, we use an early geometric approximation instead. The step-by-step algorithmic implementation for computing the TRUS feature and embedding it into the node and edge tensors of the bipartite graph is provided in [Appendix A.2](#).

6. Preliminary Methodological Investigation

Several approaches were investigated and then set aside. The limitations found helped me transition to the design of the semi-supervised and reinforcement learning approach.

6.1. Supervised Learning

The first approach was straightforward, just train the GNN using supervised learning, i.e. asking it to learn and copy the exact labels of BS status generated by the ILP solver. Note that solving the Set Cover problem means navigating a massive combination space. With 21 BS in our setup and diurnal user pool, there are millions of BS on/off combinations (2^{21}), and discussed further in the ablation studies ([Chapter 8](#)). The supervised-only model achieved an impressively low mean power consumption (434.1 W), but it also had the worst coverage rate (70.3%). Essentially, the model figured out that the trivial way to match the low-energy patterns of the training data was to simply turn too many base stations off, leading to dropping users discriminately.

6.2. Temporal GNN

Temporal Graph Neural Networks (TGNN) are advanced machine learning models designed to analyze graphs that change over time [[16](#)]. I explored using a TGNN to predict activation sequences under user mobility. However, this approach mixed two distinct and highly complex challenges: solving the SCP and managing temporal control dynamics. I found it difficult to frame, and if the model failed, it was unclear whether the error came from poor spatial coverage or from failing to anticipate user movement. Eventually, I decided to solve a static, per-snapshot coverage problem first using Graph-SCP, and delegating the temporal and mobility dynamics to a dedicated Reinforcement Learning (RL) extension (detailed in [Chapter 9](#)).

6.3. Exhaustive Search

I also evaluated a brute-force approach, which tests every single on/off combination of

base stations. As the number of combinations scales exponentially 2^N , this method is only computationally possible for tiny, trivial network setups. However, by running it on small-scale test scenarios, I was able to comprehend that the ILP solver genuinely finds the true, mathematically optimal energy configuration. Proving this on a small scale was essential, as I could confirm ILP as the oracle for generating training labels on the full 21-BS topology, where running a brute-force search is physically impossible.

6.4. Challenges Identified by the Screening

Probing these early approaches helped frame the hurdles that a UE-BS association system designed to minimize energy needs to overcome:

1. *Coverage vs. Power*: Finding the sweet spot where everyone stays connected and energy is saved, ensuring that not too many base stations are left turned on.
2. *Overfitting*: Ensuring the model doesn't simply memorize the exact answers from the limited set of ILP-generated training data.
3. *Impossible Scenarios*: Because mmWave pico and femto cell radii are small (200 m and 50 m), some snapshots are physically infeasible at 100% coverage. For example, an isolated UE might fall outside every small cell's radius and only be reachable by an expensive macro cell, or a traffic hotspot might exceed the total local capacity.
4. *High Traffic*: Closing the performance gap that appears when the network is under heavy load.

These challenges directly motivated the adoption of the semi-supervised Graph-SCP model proposed by Shafi et al. [\[12\]](#), together with a separate Reinforcement Learning (RL) extension.

7. Semi-Supervised Graph-SCP for HetNet Base Station Activation

The Graph-SCP paper by Shafi et al. [12] provided the foundation for this chapter. Building on that idea, I develop and evaluate the full adapted methodology from the ground up. It starts with how each HetNet snapshot is turned into a graph and labelled using the ILP oracle. The dataset composition, covering both hotspot and uniform traffic across a wide range of UE counts, is then introduced to show how the training data was designed to reflect real-world load diversity. From there, the chapter walks through the GraphSAGE architecture, the semi-supervised loss and its weight choices, and the training process. It then explains how inference works with the coverage repair step, and measures the final system against the ILP oracle and the greedy baseline on power, coverage, and speed. The chapter ends up with the ablation studies on loss inferences.

7.1. Design Rationale

The central insight borrowed from Shafi et al. [12] Graph-SCP is that the minimum-cost base station activation problem maps cleanly onto a weighted Set Cover Problem (SCP). The idea is to train the GNN on bipartite graph representations of UEs and BS to predict which subsets belong to the optimal cover, and later passing only those candidates to a downstream ILP solver. It provides a principled and faster alternative to full ILP enumeration.

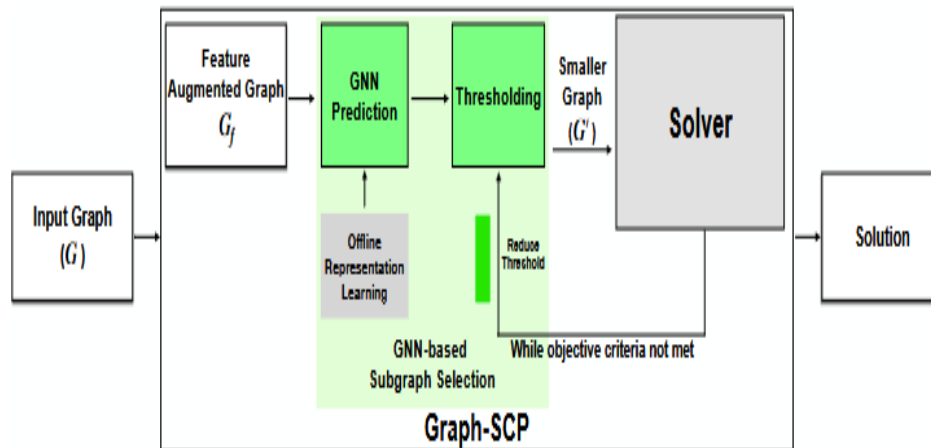


Figure 7.1: The original Graph-SCP architecture proposed by Shafi et al. [12]

While the original framework uses the GNN to reduce the search space before passing the subgraph (G) to a downstream solver like Gurobi, we adapt the pipeline for BS on/off decisions. If the thresholded prediction fails to meet the strict user coverage target, a lightweight Micro-ILP fallback is triggered exclusively to repair the remaining coverage gaps, and reach near-optimal solutions. The goal was to capture that same 10x speedup while matching the energy efficiency of the full ILP, ensuring we consistently outperform standard greedy assignments.

7.2. Dataset Composition

For the supervised labels, the full-mode ILP is solved for each instance to produce the optimal base station activation vector. Across the entire dataset of 5,680 snapshots, the ILP successfully reached 1.00 (100%) coverage without failure. As a result, the model is trained entirely on faithful, optimal solutions instead of heuristic approximations.

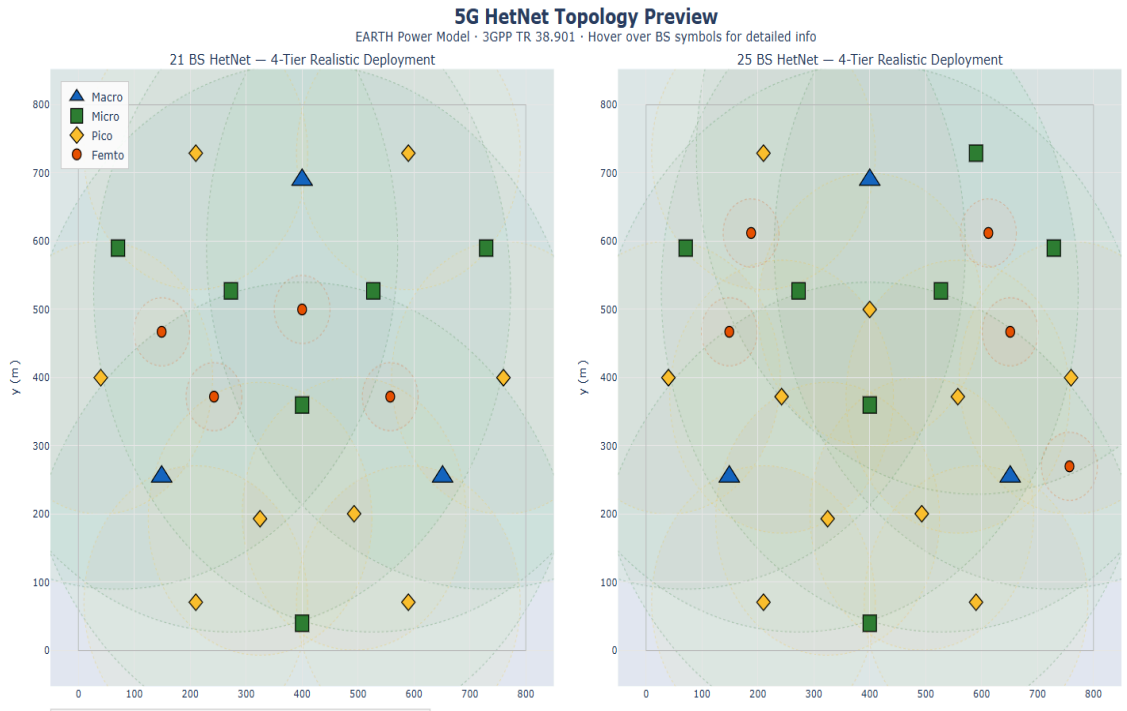


Figure 7.2: A 5G Heterogeneous Network (HetNet) topology preview illustrating a 4-tier realistic deployment. The left and right panels display spatial layouts for networks with 21 and 25 base stations, respectively, consisting of a mix of Macro, Micro, Pico, and Femto cells. The placement architecture

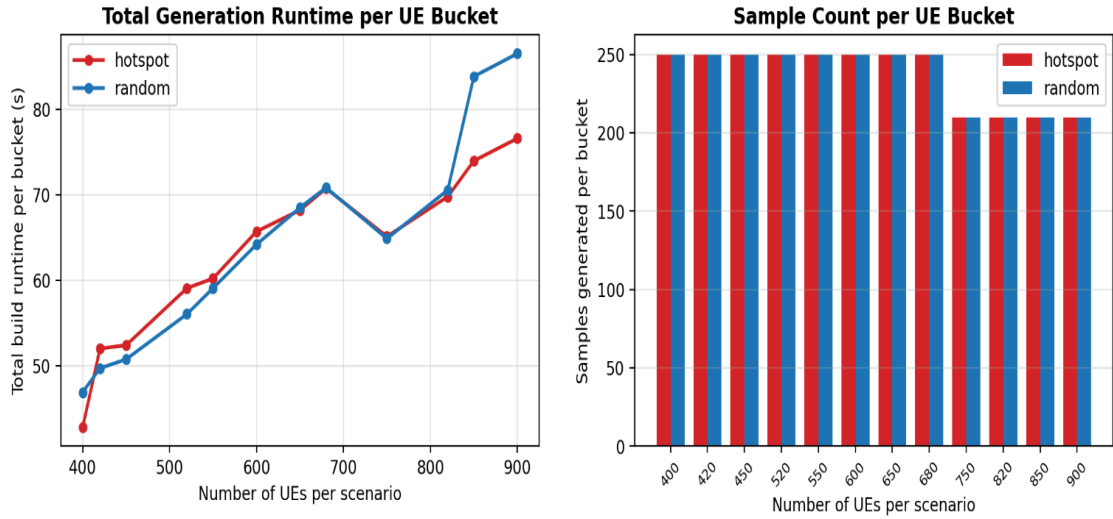


Figure 7.3: The left panel shows the total time needed to build data from the ILP solution for each user count (peaking at ~86.5 seconds for 900 users with random traffic). The right panel shows how many data samples were collected for each group (250 samples for groups up to 680 users, and 210 samples for the 750–900 user groups).

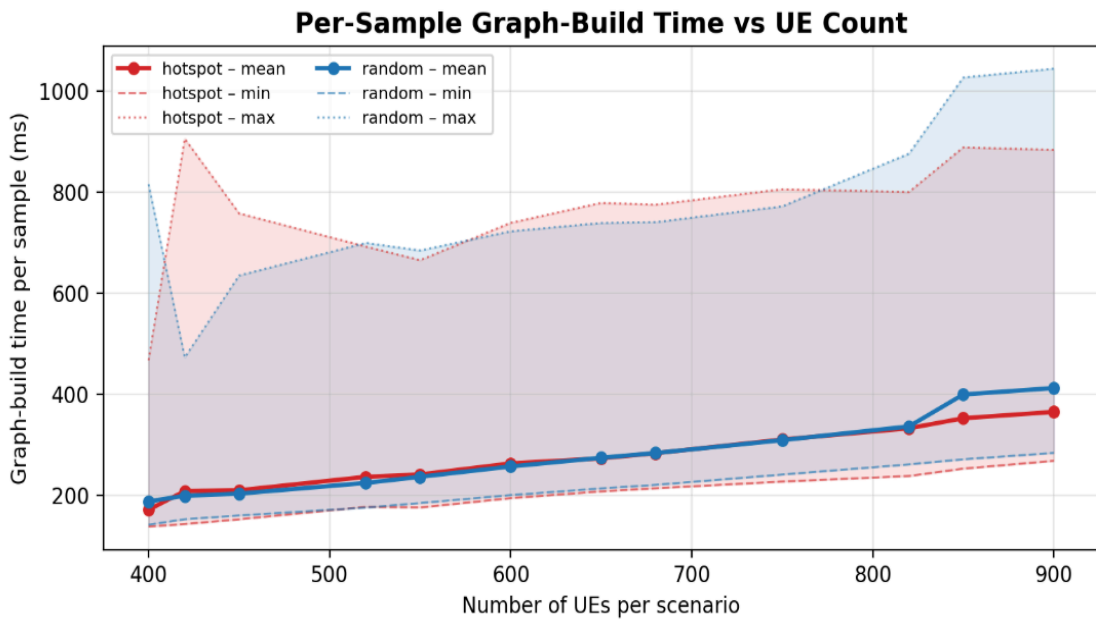


Figure 7.4: Graph-build processing time per sample across varying user (UE) counts. Solid lines show the average time for hotspot (red) and random (blue) traffic, bounded by shaded minimum and maximum ranges. Average processing time scales roughly linearly, growing from ~171 ms at 400 UEs to ~412 ms at 900 UEs, with occasional spikes up to ~1.0 s under random traffic

Generating the ground-truth labels via ILP for 5,680 training samples required over 13 hours of cumulative processing time.

7.3. GNN Construction

Each instance is encoded as a bipartite UE–BS coverage graph augmented with a universe node. Base-station nodes carry a 10-dimensional feature vector encoding domain-relevant attributes, including TRUS in slot 4 ([Chapter 5.4](#)). The edges connecting UEs to feasible BSs carry the 2-D attributes $[ce_{\text{norm}}, \text{TRUS}]$, which feed both channel quality and capacity headroom directly into the graph's connections. This constitutes the ablation condition, encoding the key insight that UE–BS connections are not of equal quality and that a BS's trustworthiness should modulate the message it contributes to the GNN aggregator.

Index	Feature Type	Feature Name	Derivation
0	Node (BS)	Normalised BS power cost	P_j^{active} / P_{max}
1	Node (BS)	Cover cardinality	$\sum_i F_{ij} / N_U$
2	Node (BS)	Random walk score (RWR)	Stationary prob, from universe node
3	Node (BS)	Node degree	$\text{deg}(j) / N_U$
4	Node (BS)	Mean neighbour degree	Average degree of element neighbours
5-6	Node (BS)	Hypergraph Laplacian features	2 nd /3 rd eigenvalues of hyperedge Laplacian
7	Node (BS)	TRUS score	TRUS _j (normalised)
8	Node (BS)	Band index (one-hot encoded)	0: macro, 1: micro, 2: small
9	Node (BS)	Sleep power ratio	$P_{\text{sleep},j} / P_j^{active}$
1	Edge (UE)	Channel quality	g_{ij} / g_j^{max} , normalised channel gain
2	Edge (UE)	TRUS proximity	TRUS replicated from the BS node

Table 7.1: Graph node and edge features for the bipartite UE–BS coverage graph

Each BS node carries a 10-dimensional feature vector encoding domain-relevant

attributes. By replicating TRUS onto the edges, the model operationalises the insight that UE–BS connections vary in quality, allowing a base station's overall trustworthiness to directly modulate the message it propagates through the GraphSAGE aggregator.

7.4. GraphSAGE Architecture

The Graph-SCP follows the GraphSAGE message-passing paradigm with GraphNorm and an edge-weighted SAGE convolution (SAGEConvEA). The architecture is a 3-layer GraphSAGE model with hidden dimension 128, ReLU activations, dropout 0.3, and batch normalisation after each layer. Formally, the layer- k update for node v is:

$$h_v^{(k)} = W^{(k)} \cdot \text{CONCAT}(h_v^{(k-1)}, \text{AGGREGATE}(\{h_u^{(k-1)} \odot e_{uv}^{(k)} : u \in N(v)\})) \quad (7.1)$$

where $e_{uv}^{(k)}$ is the edge feature vector and $\odot$ denotes element-wise scaling before the aggregation step, which is an edge conditioned message passing extension. Finally, we get scalar activation probability $p_j \in (0, 1)$ corresponding to each BS node by using a sigmoid output layer. During inference, a threshold decrement search starting at $\tau_{\text{high}} = 0.7$ (calibrated on the validation set) activates all base stations with $p_j > \tau$. If the intended coverage target is not satisfied, τ is decremented by 0.1 until a micro-ILP fallback is triggered and done. The reasons justifying to choose GraphSAGE are:

1. *Inductive scalability*: The sample-and-aggregate operator allows a model trained on 420-900 UEs to transfer seamlessly to 175–1,000 UEs without retraining (essential for the OOD evaluations in Chapter 9.3).
2. *Variable graph sizes*: GraphNorm [11] stabilises activations across graphs whose size varies almost three-fold, where batch statistics would be meaningless since each batch is a single large graph.
3. *Domain modulation*: the edge-weighted convolution lets the channel - TRUS edge attributes shape message passing, so the domain features actually influence the computation rather than being inert metadata. Semi-supervision then anchors the model to the ILP labels while physics-based losses prevent label-only overfitting.

7.5. Semi-Supervised Loss Function

The loss function critically contributes to differentiate the approach from a purely supervised and purely unsupervised baseline. I adapted the Graph-SCP objective to our energy minimisation plan, and the loss for training instances is given as:

$$L = \lambda_{sup} \cdot L_{BCE}(p, y^*) + \lambda_{cost} \cdot p^T c + \lambda_{cov} \cdot ||(Ap - 1)_+||_1 + \lambda_{sp} \cdot (1 - Ap)_+^T 1$$

where $L_{BCE}(p, y^*) = -[y \log p^* + (1 - y) \log(1 - p^*)]$

- $y^* \in \{0, 1\}^{N_B}$ is the ILP-optimal activation label vector
- L_{BCE} is the element wise binary cross-entropy
- $c \in R^{N_B}$ is the vector of normalised BS operating power costs
- $A \in \{0, 1\}^{N_U \times N_B}$ is the feasibility matrix
- $(\cdot)_+$ denotes the ReLU (positive part), which penalises under-coverage of users without penalising the over-coverage
- L_{BCE} is the element-wise binary cross-entropy
- $p^T c$ is the power cost penalisation
- $|| (Ap - 1)_+ ||_1$ is the coverage constraint satisfaction
- $p^* \in [0, 1]^{N_B}$ is the predicted BS activation probability
- $y \in \{0, 1\}^{N_B}$ is the ILP optimal binary label

The supervised term anchors the predictions to the quality solutions, while the unsupervised term generalised beyond them by directly minimising the physical objective. The third term encourages sparse activation, like GNN learns to select fewer base stations while still covering all users.

7.6. Graph-SCP Pipeline

The end-to-end pipeline discussed in the preceding sections is summarized as follows:

- A. Real-time snapshot of a four-tier HetNet, comprising 420–900 UE(UEs) distributed across 21 base stations (Macro, Micro, Pico, Femto).
- B. The snapshot is translated into a directed Set-Cover graph $G=(V,E)$. It features a central universe node, intermediate UE element nodes, and 21 base station nodes, enhanced with the calculated TRUS metric.
- C. A 3-layer GraphSAGE neural network processes this graph using semi-supervised learning to output a selection probability $P(b_j)$ for each base station.
- D. During inference, a threshold-decrement search incrementally lowers the activation threshold τ (from 0.70 to 0.0) to activate base stations until the strict 99.5% coverage target is satisfied. If the target is still unmet, a micro-ILP fallback algorithm guarantees coverage by repairing the final active base station mask.

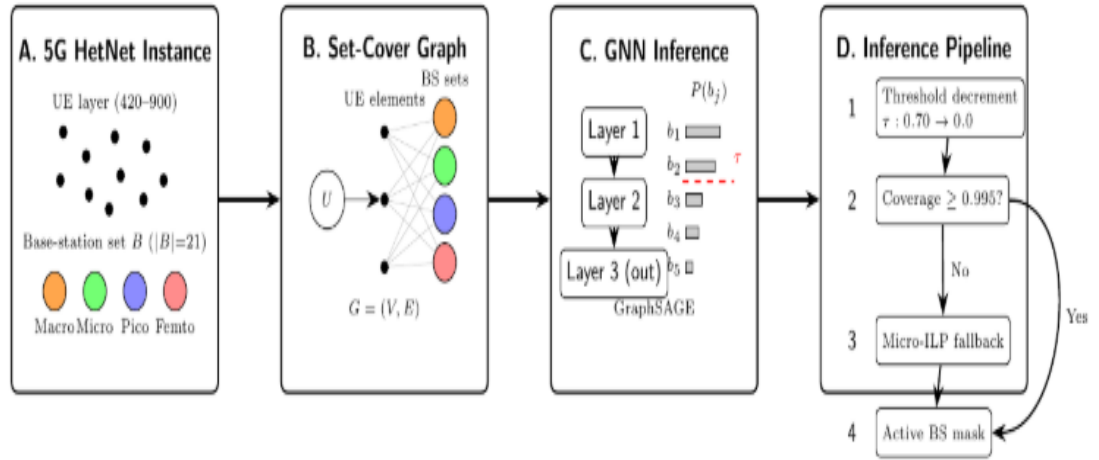


Figure 7.5: TRUS-augmented Graph-SCP pipeline for 5G HetNet energy minimisation (adapted from Shafi et al., 2023)

7.7. Training and Convergence

The model was trained using the AdamW optimiser and a CosineAnnealing scheduler, employing early stopping to prevent overfitting. Training halted at epoch 37, with the best model checkpoint recorded at epoch 22, achieving a minimum validation loss of 0.2886. Over the 37 epochs, the total training loss declined smoothly from 0.3014 to

0.1819, driven primarily by the supervised loss component. The computational overhead was highly efficient, maintaining a mean runtime of 43.39 seconds per epoch and completing the entire training process in approximately 26.8 minutes. Full hyperparameter configurations are detailed in [Appendix C.1](#).

Metric	Value
Total Dataset Size	5680 instances (4828 train / 852 val)
Total Training Time	~26.8 minutes
Training F1 score	0.9182
Training Accuracy	0.9068

Table 7.2: Training convergence summary

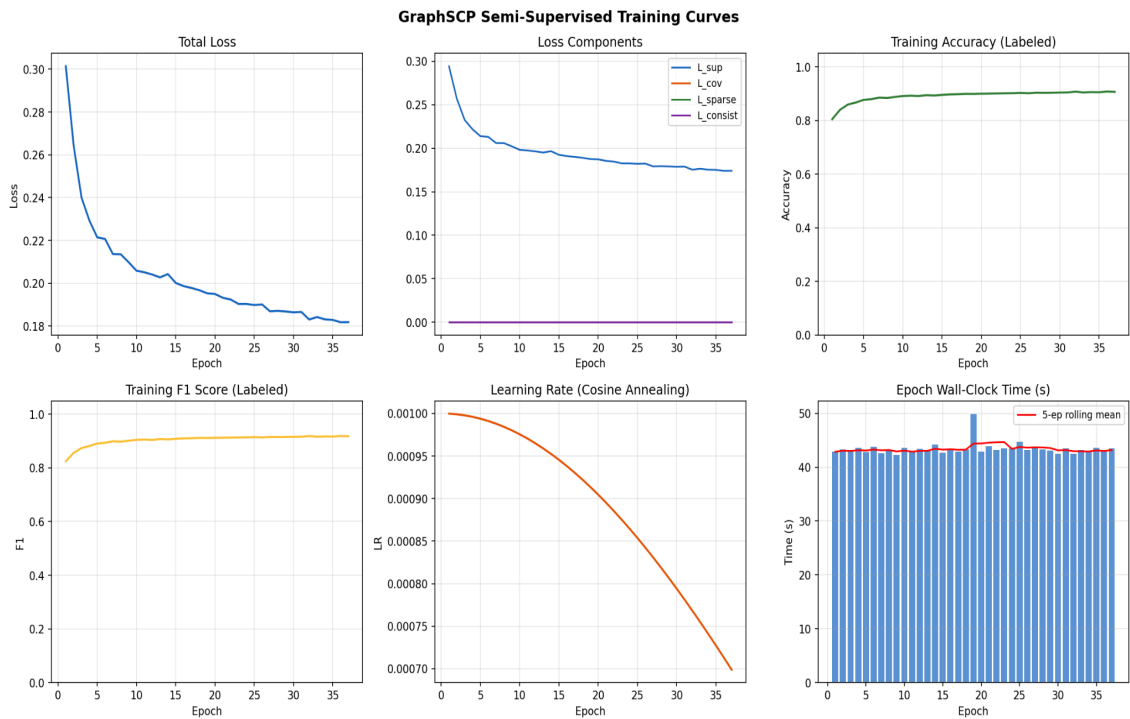


Figure 7.6: GraphSCP semi-supervised training curves over 37 epochs. The plots illustrate a smooth decline in total and supervised loss, rapid stabilisation of accuracy and F1 scores, a properly decaying learning rate, and consistent computational runtimes averaging ~43 seconds per epoch.

7.8. Evaluation: Power, Coverage and Optimality Gap

As mentioned earlier, three algorithms are evaluated head-to-head on each test instance:

1. **ILP (exact):** CBC solver with 60 s time limit and optimality gap constraints (ground truth)
2. **Graph-SCP (novel proposed):** GNN prediction followed by threshold-decrement inference and micro-ILP fallback if needed
3. **Greedy set cover:** Iteratively activates the BS with the greatest marginal UE coverage, tie-breaking by lowest power cost.

The adapted Graph-SCP was evaluated on 60 held-out test instances. Eight of these were physically infeasible, i.e. the requested coverage of 99.5% on ILP simply could not be met given the deployed base stations, and were excluded. **This left 52 feasible instances spanning 420 to 1,000 UEs across both random and hotspot traffic.**

Notably, 6 out of these 52 instances contained user counts that the model never saw during training. All comparisons below are based on these 52 instances, using the exact ILP solution as the reference.

Method	Mean Power (W)	Mean Coverage	Mean time to predict active BS mask (s)	Power Savings vs Greedy	Power consumption vs ILP
ILP Oracle	786.8	100%	5.950	48.5% saved	-
Graph-SCP	1150.4	98.8%	0.745	28.58% saved	42.3% more power
Greedy Baseline	1527.5	99.84%	0.072	-	94.1% more power

Table 7.4: Aggregate comparison on the 52 feasible test instances on power, coverage, prediction time

Graph-SCP delivers **$\sim 8\times$ average speedup** over the exact ILP (0.745 s vs 5.95 s), and stays within +46% of the optimal power, thereby landing in the sweet spot between the slow-but-optimal ILP and the fast-but-wasteful greedy baseline. It recovers roughly half the ILP's savings while remaining near-real-time, and the speedup is largest precisely on the dense high-load instances (up to $\sim 86\times$) where the ILP becomes intractable. Meanwhile, greedy association has approximately $83\times$ speedup, but at the

price of +94% overhead. This is simply shown in the plot below:

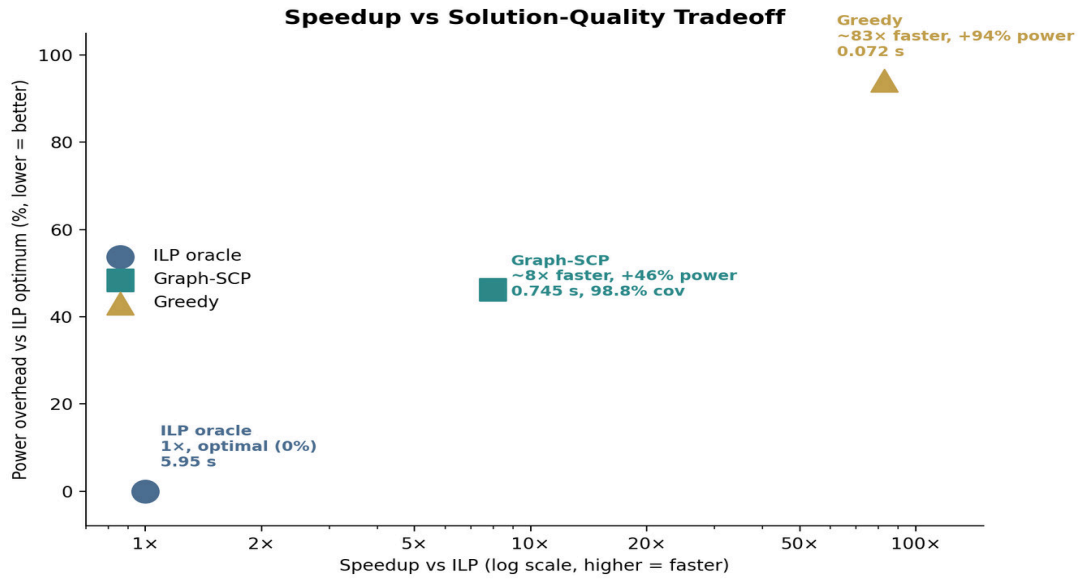


Figure 7.7: Speedup vs Power Optimization across the three algorithms. Although Graph-SCP uses 46.21% more power than the mathematically optimal ILP, it compensates by solving the network configuration nearly 8 times faster (0.745s), all while maintaining a high mean coverage of 98.8%.

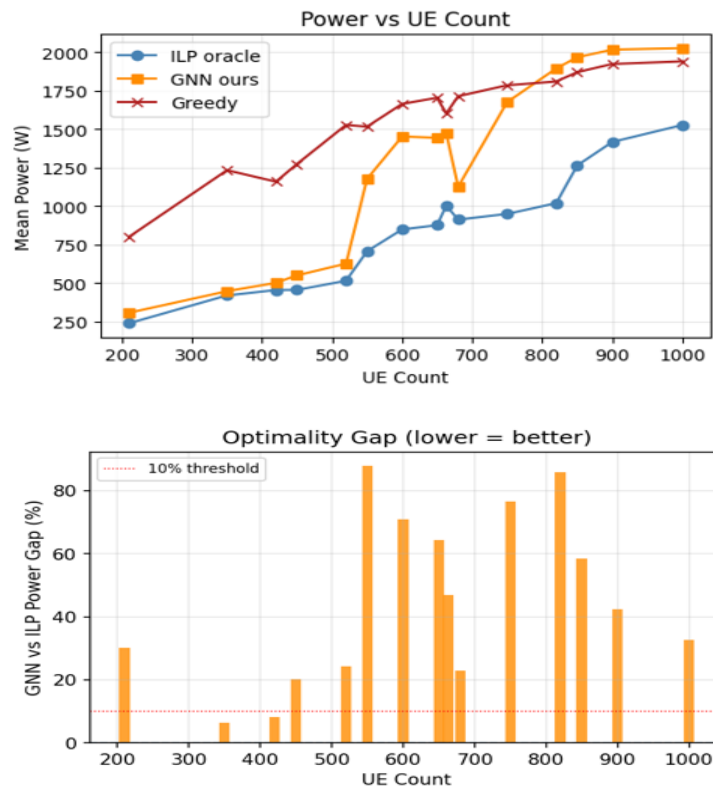


Figure 7.8: Evaluation of Graph-SCP against the ILP Oracle and Greedy baseline on the feasible test set. (Top) Mean power consumption versus UE count. (Bottom) The optimality gap in power consumption, showing increased divergence during high-load

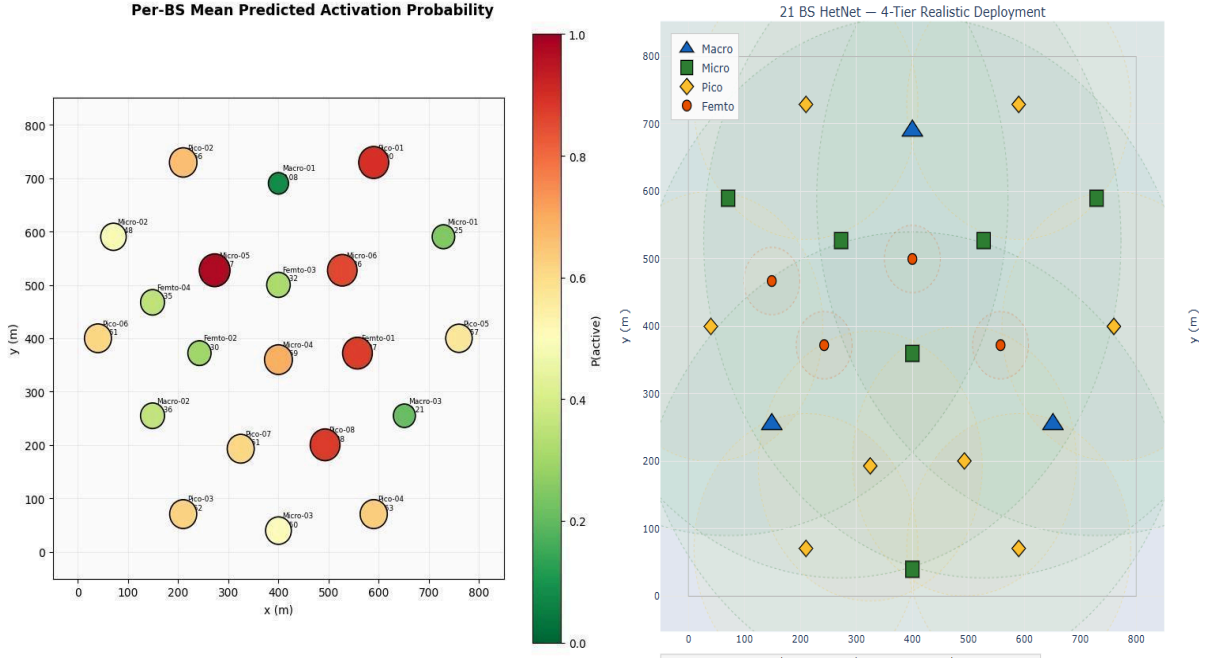


Figure 7.9: (Left) Activation decision across 21-BS HetNet topology. Circle colour indicates activation frequency (red = activated almost always; green = mostly kept in sleep mode). (Right): HetNet topology for tier reference

The colour pattern is clearly non-uniform and spatially coherent. It is also centrally located. High-utility base stations such as Pico-01, Micro-05, Micro-06, and Femto-01 are activated almost always red, i.e. $P = 1.0$, confirming that the model correctly identifies these as essential for maintaining coverage under varying user loads. In contrast, peripheral and geometrically redundant cells, including the 3 Macros and edge Pico nodes are routinely left in sleep mode, reflecting the model's learned ability to exploit spatial overlap and suppress unnecessary activations. This structured activation pattern, rather than uniform switching, directly demonstrates that Graph-SCP has learned the underlying topology of the HetNet and is making energy-aware decisions that closely mirror the logic of the ILP oracle.

7.9. Instance-Level Deep Dive

To move beyond aggregate statistics, I am examining two representative evaluation instances in depth, a best observed case and a worst case, and ponder into when and why Graph-SCP succeeds or struggles.

7.9.1. Best Case: Graph-SCP Matches the Oracle

Evaluation instance #4 is a random-traffic scenario with 350 UEs across the 21-BS topology. Graph-SCP matched the ILP oracle almost exactly, 364 W vs. 365 W, with 100% coverage for both, while Greedy consumed nearly three times as much power (1,079 W).

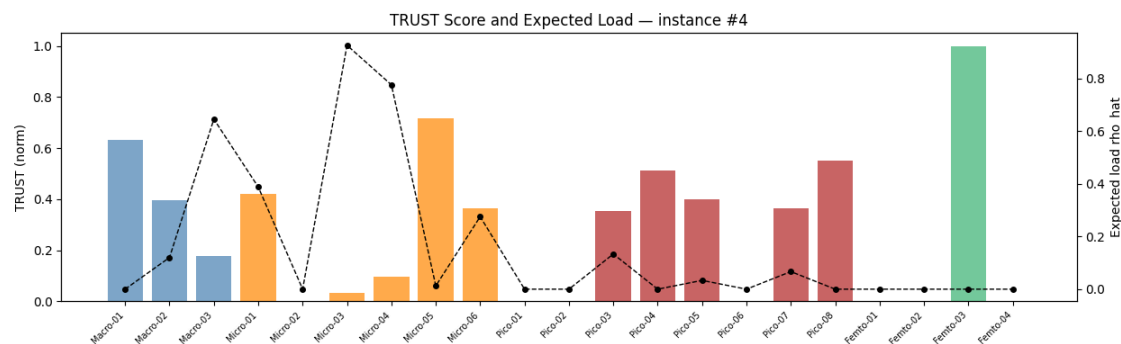


Figure 7.10: TRUS score (bars) and expected load (dashed line) per base station for Instance number 4. High TRUS and high load rarely coincide, validating the use of both metrics as complementary graph features.

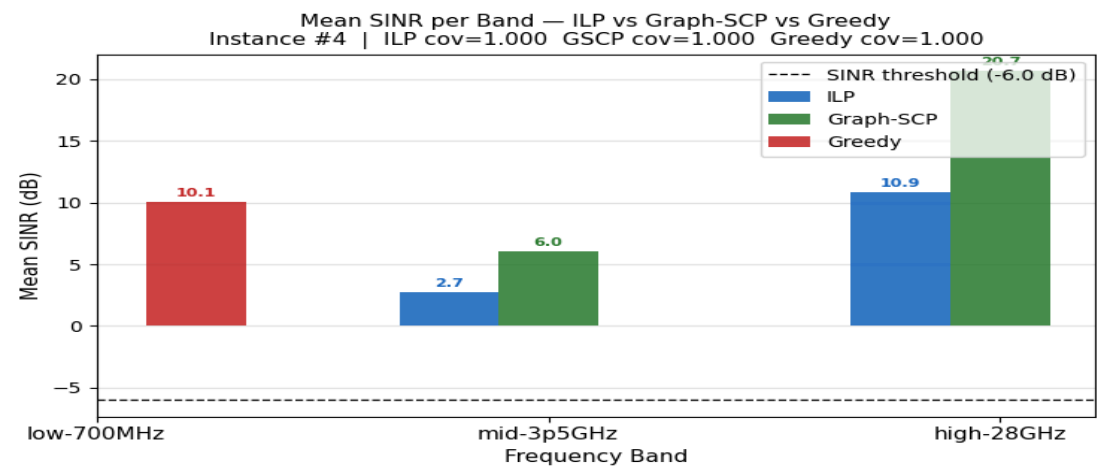


Figure 7.11: Mean SINR per frequency band for ILP, Graph-SCP, and Greedy. Graph-SCP achieves the strongest signal quality across mid and high bands, while Greedy concentrates power wastefully in the low-700 MHz band.

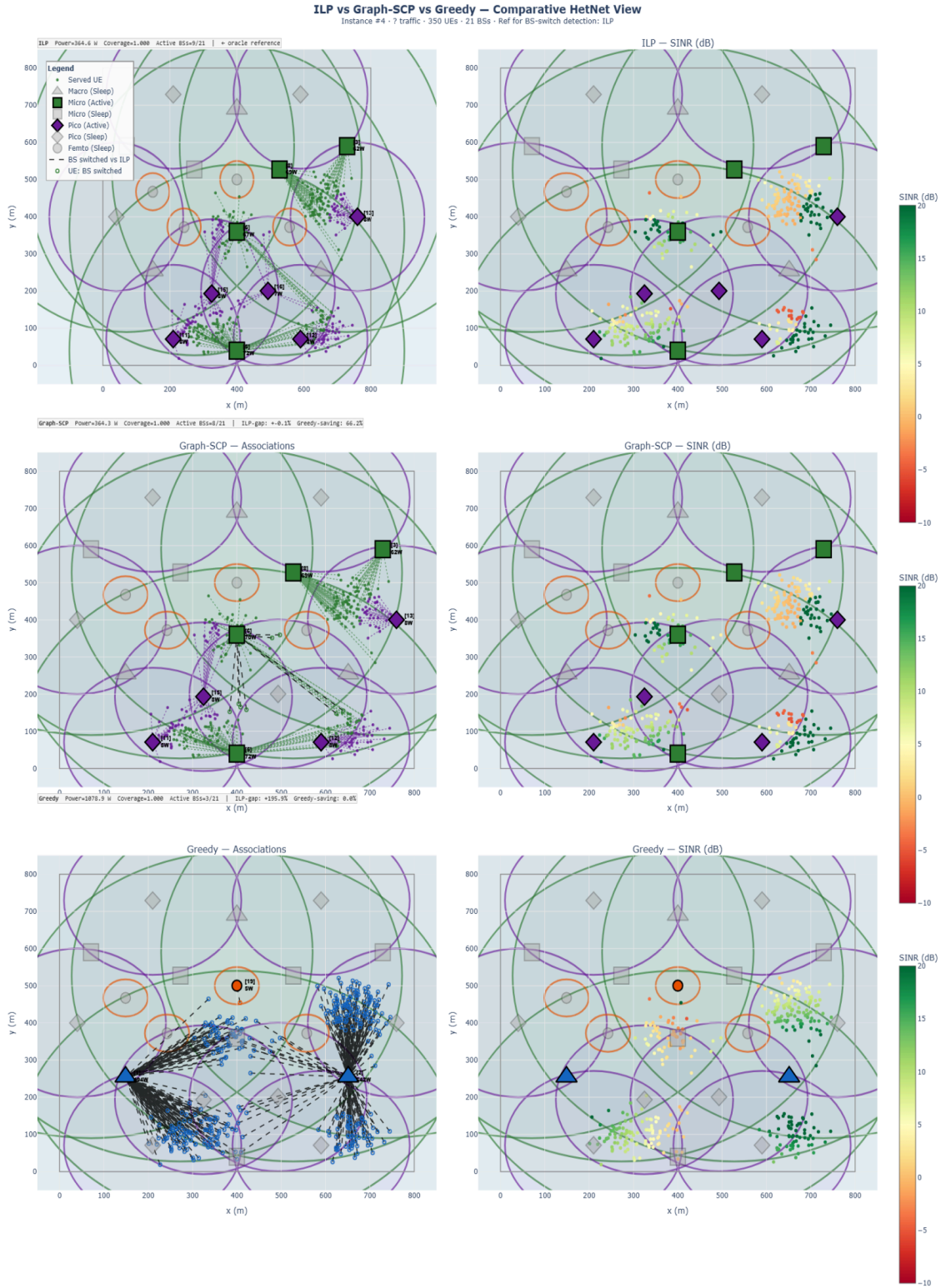


Figure 7.12: Comparative spatial view of UE–BS associations (left column) and per-user SINR maps (right column) for ILP, Graph-SCP, and Greedy on Instance 4 (350 UEs, random traffic). ILP and Graph-SCP produce near-identical, efficient associations with strong SINR across the coverage area, while Greedy funnels all users toward three Macro towers at significantly higher energy cost.

7.9.2. Worst Case: Graph-SCP Failure Analysis

Evaluation instance 11 is a hotspot-traffic scenario with 900 UEs, the second highest load configurations in the test set. This is where Graph-SCP is at its worst, consuming 2,025 W against the ILP's 1,411 W, a 43.5% optimality gap. All three methods achieve 100% coverage, but differ in how they do it.

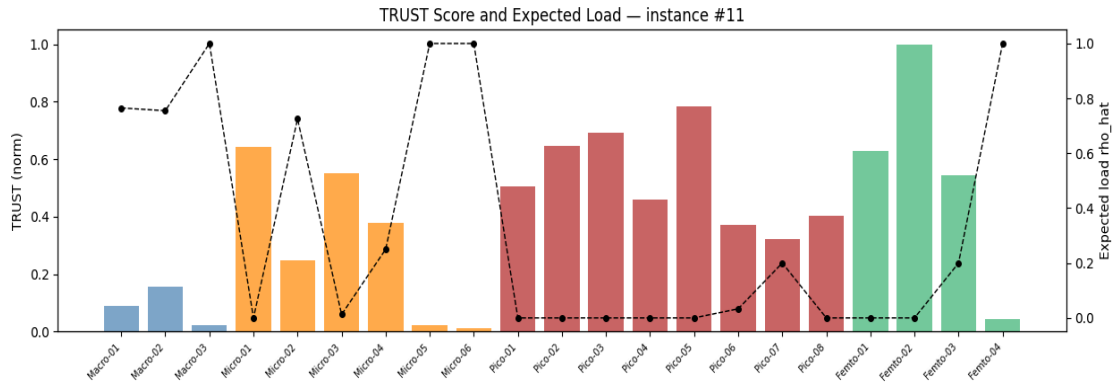


Figure 7.13: TRUS score (bars) and expected load (dashed line) per base station for Instance number 11. The TRUS score is a property of the base station and its channel conditions, it does not change with the number of users. This combination, high load everywhere, no clear low-load cells to switch off is precisely what makes this instance hard

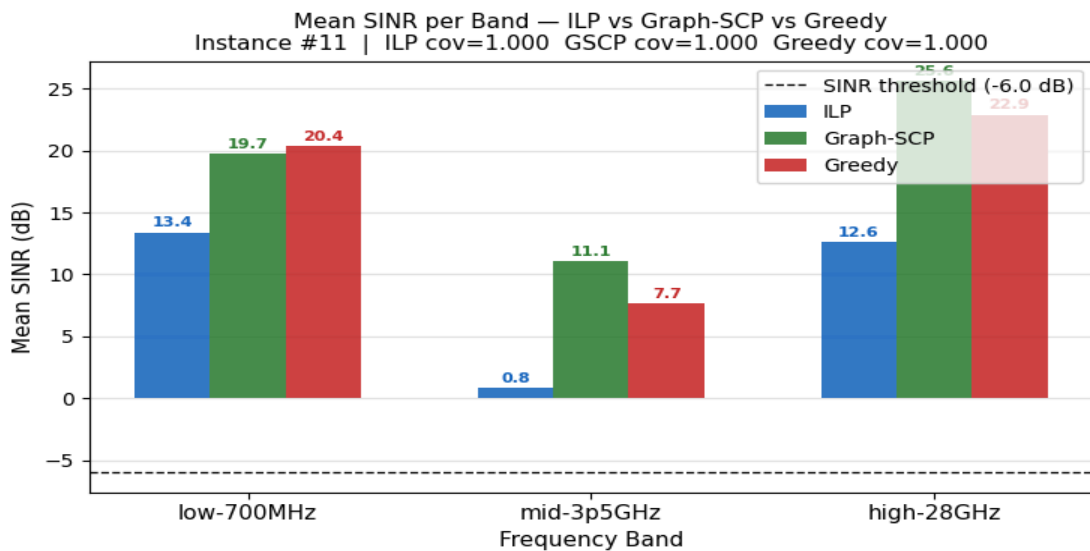


Figure 7.14: Mean SINR per frequency band for ILP, Graph-SCP, and Greedy. Graph-SCP achieves the highest signal quality due to over-activation, confirming that its energy penalty is not a coverage failure but an unnecessary power expenditure.

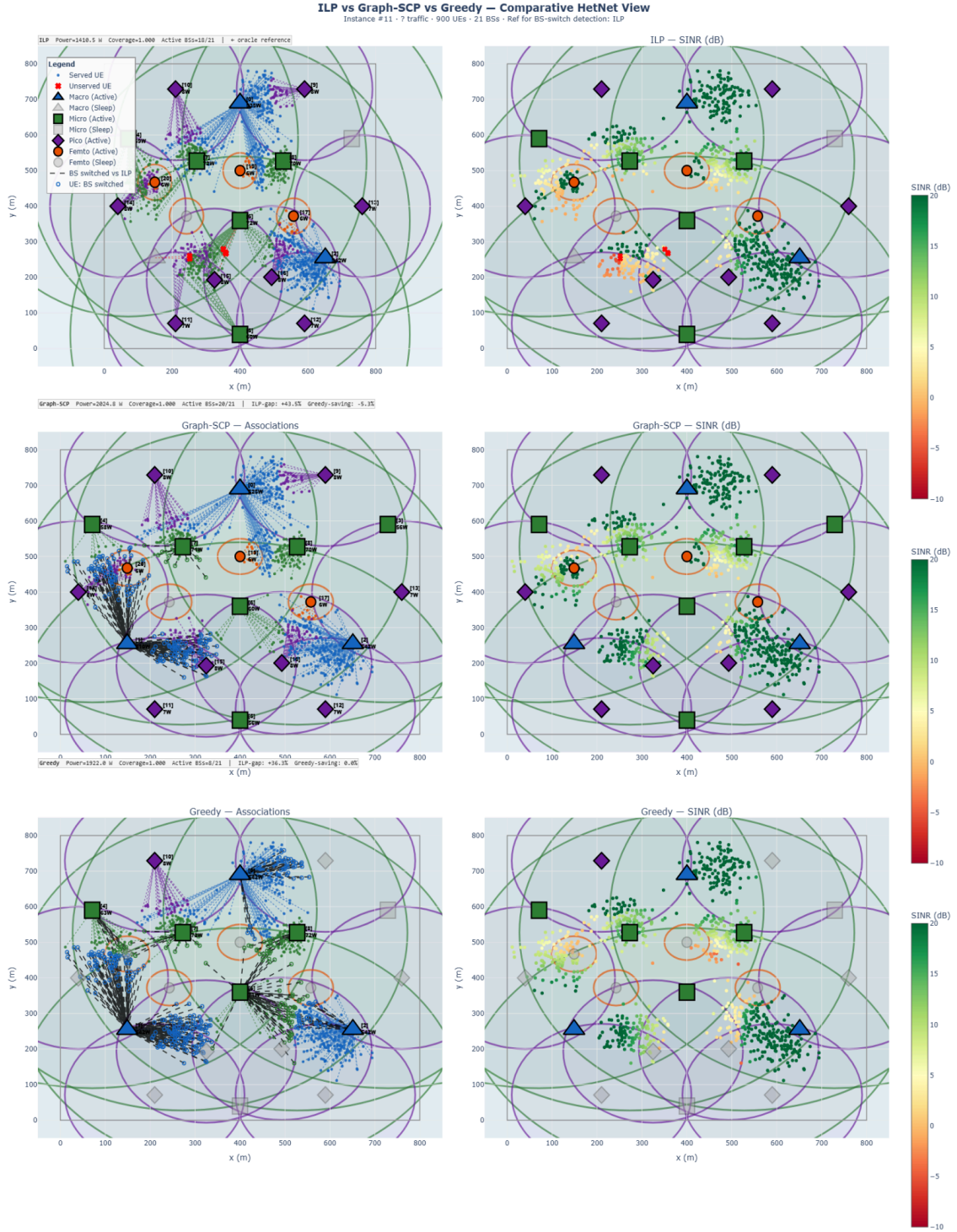


Figure 7.15: Comparative spatial view of UE–BS associations (left column) and per-user SINR maps (right column) for ILP, Graph-SCP, and Greedy on Instance 11 (900 UEs, hotspot traffic).

In the ILP panel, there is a small cluster of edge UEs that are cleanly picked up by an additional low-power cell (a Pico/Femto) which the oracle chooses to keep active. This extra BS is precisely what prevents those users from sitting on the SINR cliff: they appear as well-served, green points in the ILP

In the ILP panel, there is a small cluster of edge UEs that are cleanly picked up by an additional low-power Pico or Femto which the oracle ILP chooses to keep active. This extra BS is exactly what prevents those users from sitting on the SINR cliff. They obviously appear as well-served, green points in the ILP SINR map. In contrast, Graph-SCP and the Greedy baseline both switch this cell off, so the same users are either dragged onto a distant Macro and left right at the coverage threshold. The global coverage metric still reports 100%, but ILP quietly wins on these boundary cases by paying a tiny extra power cost to avoid fragile, near-outage UEs that the learned and greedy policies fail to protect explicitly.

Please note: The target coverage taken for the ILP constraint is 0.995 as mentioned earlier, and the 100% coverage shown on the top left is simply a rounded value, not to be confused

8. Ablation Study of TRUS within Graph-SCP

8.1. Ablation Design

To understand TRUS and the semi-supervised loss shape performance, I ran a four-condition ablation on the validation set (n=852 instances). Each condition trains a fresh GraphSCP model from scratch for up to 50 epochs under identical hyperparameters, differing only in which TRUS-related components are included:

Configuration	TRUS as node feature (Slot 4)	2-D edge features	Semi-supervised loss
Supervised only	No	No	Supervised BCE only (equation)
trus_slot_4	Yes	No	Full
TRUS as edge feature	No	Yes	Full
Full Loss	Yes	Yes	Full

Table 8.1: Ablation Study Configurations

This design allows isolating the effect of TRUS as a node feature, the incremental gain of 2-D edge attributes and the combined benefit of both.

8.2. Ablation Results

The ablation bar chart illustrates the three evaluation metrics across the four conditions. Quantitative results are as follows:

Config	Best Loss	Mean Power (W)	Best coverage rate	Label accuracy (top-k)
Supervised only	0.4048	434.1	0.7033	0.7411
trus_slot_4	0.239	697.1	0.9306	0.8877
TRUS as edge feature	0.5548	411.9	0.7096	0.7186
Full Loss	0.2487	639.5	0.9082	0.8737

Table 8.2: Ablation isolating TRUS (validation set, n = 852)

On inferring the table [8.1](#), Node-level TRUS (trus_slot_4) and the full model are the only configurations that hold decent coverage. The low-power configurations achieve their power by under-covering. Note that supervised_only loss is trained without the micro-ILP fallback trigger.

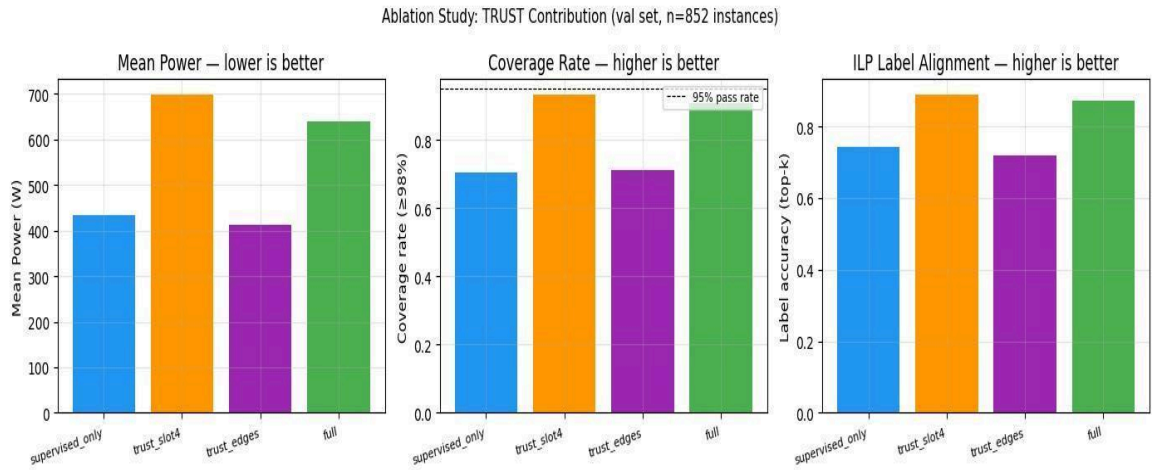


Figure 8.1: Three-panel ablation of the TRUS contribution

Several findings stand out:

- 1. TRUS node feature reduces validation loss by ~16%.** This quantifies the contribution of the Voronoi-proportional TRUS score to the GNN's ability to discriminate high quality BS activations from poor ones. It proves the GNN needs a node-level summary of capacity and channel quality to confidently keep necessary base stations active, which naturally pushes the power up to a realistic operational level
- 2. 2-D edge features contribute independently.** When TRUS is only provided on the UE-BS edges, the GNN struggles to aggregate it properly. It results in the worst validation loss (0.5548) and the worst label accuracy (71.8%). It needs the node-level context to make sense of the edge data.
- 3. The Full Model hits the sweet spot.** It deliberately trades a tiny margin of coverage and accuracy (dropping to 90.8% and 87.3%) in order to pull the mean power down by nearly 60 watts (from 697.1 W to 639.5 W). It learns to safely shave off excess energy without crossing the line into dangerous under-coverage.

Thus, a clear hierarchy of contribution is revealed from the ablation study.

TRUS as node feature > TRUS as edge feature > Supervised Loss.

8.3. Remaining Challenges of Graph-SCP

Developing Graph-SCP ultimately centered on two major challenges.

1. Tuning the semi-supervised loss to guarantee coverage without wasting power.
2. A persistent challenge was the model's optimality gap under high load, where it struggles to efficiently choose between overlapping Macro towers and dense small cells. This gap isn't a bug, but a fundamental limit of relying on a fast heuristic for the hardest instances.
3. The most important was that the GSCP evaluates the network as a static snapshot, and cannot natively track temporal dynamics and model the complex interference caused by user handovers between time steps.

9. Reinforcement Learning as Baseline: PPO for HetNet

So far, the semi-supervised model only solves static snapshots. But real networks are dynamic. Users move, traffic shifts, and toggling cell connections results in handover costs. This led me to explore a reinforcement learning-based controller [18]. I want to clarify beforehand that this approach is strictly an exploratory proof of concept, presenting preliminary data to show the logical next step for handling moving networks.

9.1. Markov Decision Process Formulation

To model the dynamic nature of the HetNet, it is imperative to frame the decision problem as a Markov Decision Process (MDP). Here, the controller operates within a custom environment, HetNetEnergyEnv, which reuses the same 21-BS Hetnet topology and radio model from the G-SCP approach, aiming for a target coverage of 0.995.

The MDP is defined as the tuple:

$$M = (S, A, P, r, \gamma) \quad (9.1)$$

The core components are structured as follows:

- **State Space**

The state at a given time step, $s_t \in R^{214}$, gives a snapshot of the network's current condition. It includes the status, load, and recent activation history for each of the 21 BS, alongside aggregate coverage metrics and the current traffic and mobility context. This gives the agent information about each cell individually, as well as an overall view of the network.

- **Action Space**

The action a_t determines the activation or sleep state for each BS in the upcoming time step. Because deciding the state of 21 base stations creates a massive combinatorial space, I opted to use a per-base station encoder with shared parameters rather than a simple flat neural network head. This allows the model to scale and learn base station behaviors more effectively.

- **Transitions and Reward Formulation:** The transition probability given by

$P(s_{t+1}|s_t, a_t)$ is driven by UE mobility. I modeled it using a random waypoint model with three different user classes and the existing underlying radio characteristics. The reward function is r_t , and is crucial because it balances the major objectives like maintaining the coverage, saving energy, and minimizing disruptive handovers. The weighted sum of r_t is given as follows:

$$r_t = w_C r_C(s_t, a_t) + w_E r_E(s_t, a_t) + w_H r_H(s_t, a_t) \quad (9.2)$$

Here:

r_C rewards the system for approaching the 0.995 coverage target and heavily penalizes any shortfall.

r_E provides a positive signal for keeping network power low, normalized against the maximum possible power ($P_{\max}$).

r_H applies penalties for handovers, specifically ‘ping-pong’ oscillations where a user rapidly bounces between cells.

Based on the convergence traces from my initial runs, the energy reward r_E hovered between -0.5 and -0.8, the coverage penalty c_E between -2 and 0, and the handover penalty between -1 and -1.3

Ultimately, the agent’s goal is to maximise the expected return G_t , with the value function given as:

$$G_t = \sum_{k \geq 0} \gamma^k r_{t+k} \quad (9.3)$$

$$V_\phi(s_t) \approx E[G_t | s_t] \quad (9.4)$$

I used a discount factor of $\gamma = 0.99$ to ensure the agent prioritizes long-term stability and sustained energy savings over immediate short-sighted cell toggling.

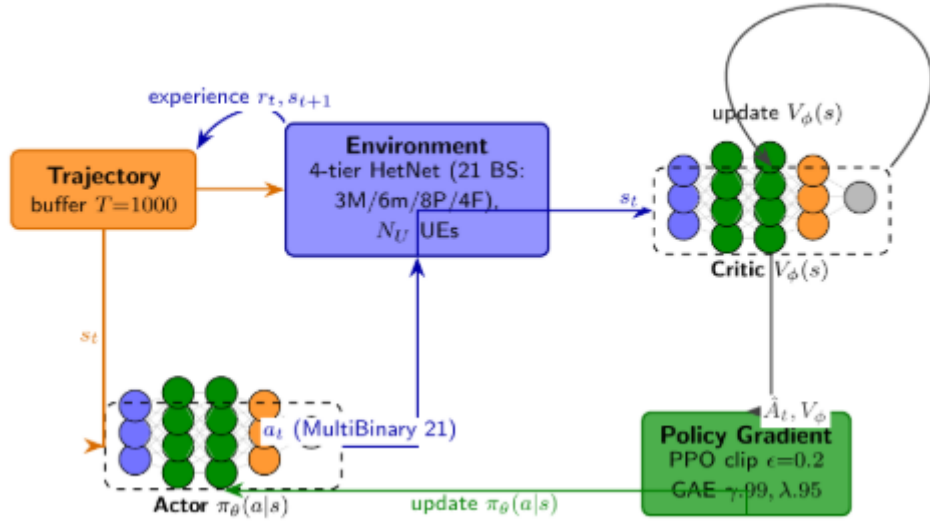


Figure 9.1: PPO actor-critic MDP loop for the 4 tier HetNet BS activation

The diagram reiterates the actual PPO learning loop. The environment feeds experience tuples, incorporating the base stations' state machine (OFF / WARMING / ON) into a 1000 step trajectory buffer. During each step, the state s_t drives the actor network (π_θ) to output the binary activation decisions, while the critic network (V_ϕ) simultaneously estimates the state's value. Using these value estimates, the PPO algorithm computes Generalized Advantage Estimation (GAE- λ) with $\gamma = 0.99$ and $\lambda=0.95$. Finally, the model updates its policy using PPO's clipped surrogate objective ($\epsilon=0.2$) to progressively optimize the energy-coverage trade-off.

9.2. Proximal Policy Optimisation and Total Loss

To train the agent, I used Proximal Policy Optimization (PPO) alongside Generalized Advantage Estimation (GAE) and a slowly decreasing exploration rate, also called as entropy annealing [10].

First, GAE calculates the advantage $\hat{A}_t$, which helps the agent figure out if an action turned out better or worse than expected. This smooths out the learning signal over long episodes as:

$$\delta_t = r_t + \gamma V_\phi(s_{t+1}) - V_\phi(s_t) \quad (9.5)$$

$$\hat{A}_t = \sum_{l \geq 0} (\gamma \lambda)^l \delta_{t+l} \quad (9.6)$$

To update its strategy safely, PPO compares the new policy against the old one using a probability ratio, $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$. It clips this ratio so the agent does not change its behaviour drastically in a single step:

$$L_{clip}(\theta) = E_t [\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t)] \quad (9.7)$$

Finally, the loss function to balance policy update, estimation errors and entropy bonus is given by:

$$L_{PPO} = -L_{clip}(\theta) + c_v L_{value}(\Phi) - c_e L_{entropy} \quad (9.8)$$

The above step is crucial for BS, as the clipping forces the model to learn in small and safe steps. Without it, a single bad update might turn off too many cells at once and completely ruin network coverage. I empirically tweaked the entropy weight during the later training stage so that the exploration phase drops and the agent can lock into a reliable energy saving setup.

9.3. Mobility and Handover Modelling

To realistically capture the cost of BS sleeping, I modeled user mobility using a random-waypoint model and implemented standard A3 event handover mechanics [17]. A handover is triggered when a neighboring cell's signal exceeds the serving cell's signal adjusted for hysteresis and offsets for a sustained Time-To-Trigger (TTT) window:

$$M_{nbr} - Hys \geq M_{serv} + Off \quad (9.9)$$

Explicitly modeling handovers is crucial. If the agent aggressively turns off cells to save power, it leads to forced frequent handovers, including rapid ‘ping-pong’ oscillations between the cells, thereby degrading the UE’s Quality of Service (QoS). This tension is the core trade-off the RL reward function must balance.

Here, I split handovers into two categories. The first is mobility handovers. They are unavoidable and caused by UEs naturally moving across boundaries. The second is the policy handovers forced by the controller switching off a cell while the user is stationary. Because policy handovers are a controllable harm, the agent should be

heavily penalized for them.

Currently in my experiment, the logs showed a persistent handover penalty (r_H) between -1.0 and -1.3, along with an elevated rate of 1.855 handovers per UE. It simply means the agent has not yet learned to minimize its policy-induced handovers, and this would be a future direction to tune constrained rewards.

9.4. Policy Architecture and Hyperparameters

The PPO actor-critic model uses a per-base-station encoder and a shared trunk, with 421,654 trainable parameters in total. The observation space is kept straightforward by directly replicating the feature set already established for the GSCP model, ensuring the RL agent operates on the exact same rich, normalized state representation. Since the observation space is 214-dimensional and the network contains 21 base stations, the 20,000-step training budget is still relatively small. For that reason, I treat these results as early evidence of learning rather than full convergence. The complete hyperparameter configuration for the PPO agent is provided in [Appendix C.2](#). Also, it took ~6.5 hours to run the PPO training in the Colab environment.

9.5. Training Behaviour

PPO Agent Convergence Metrics Over Training Timeline (Seed: 42)

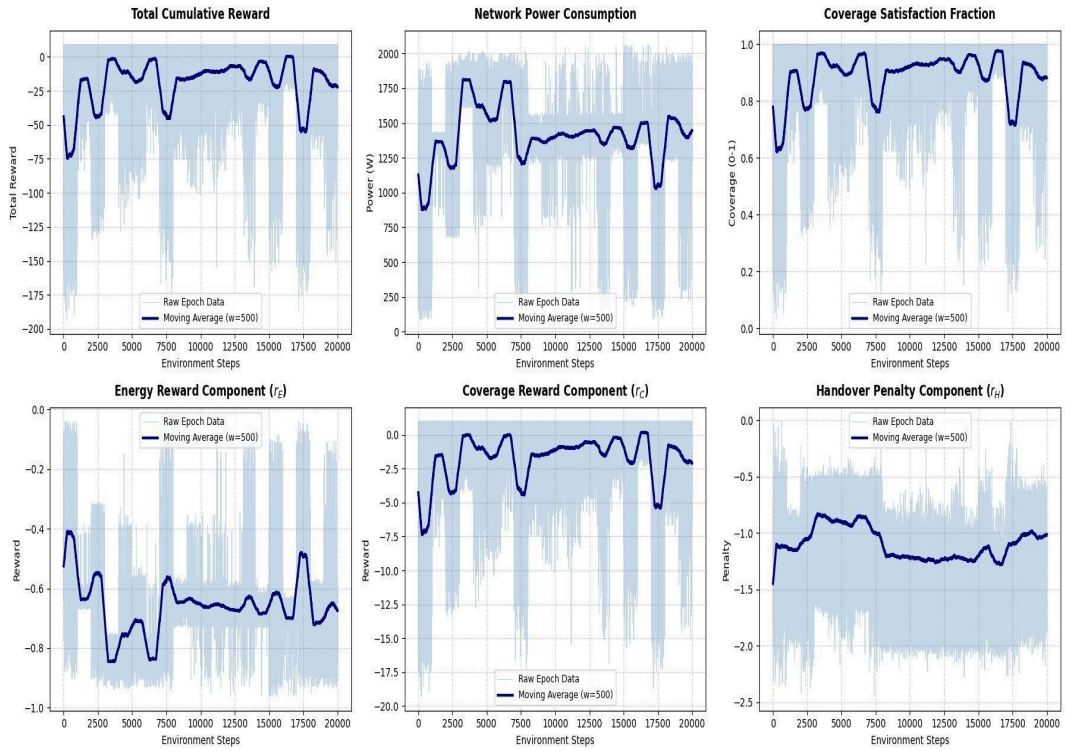


Figure 9.2: PPO Agent Convergence Metrics (in clockwise) - Total cumulative reward, network power, coverage fraction, and individual reward components (r_E , r_C , r_H) over 20,000 training steps

From Figure 9.2, we could see that the total reward is volatile, and its moving average clearly rises. The agent successfully stabilizes network power around 1300–1500 W, meaning it has learned an aggressive energy-saving policy. However, coverage still fluctuates between 85% and 95%, with recurrent dips. This indicates that while the agent knows how to cut power, it has not yet fully learned how to reliably guarantee the strict 99.5% coverage target.

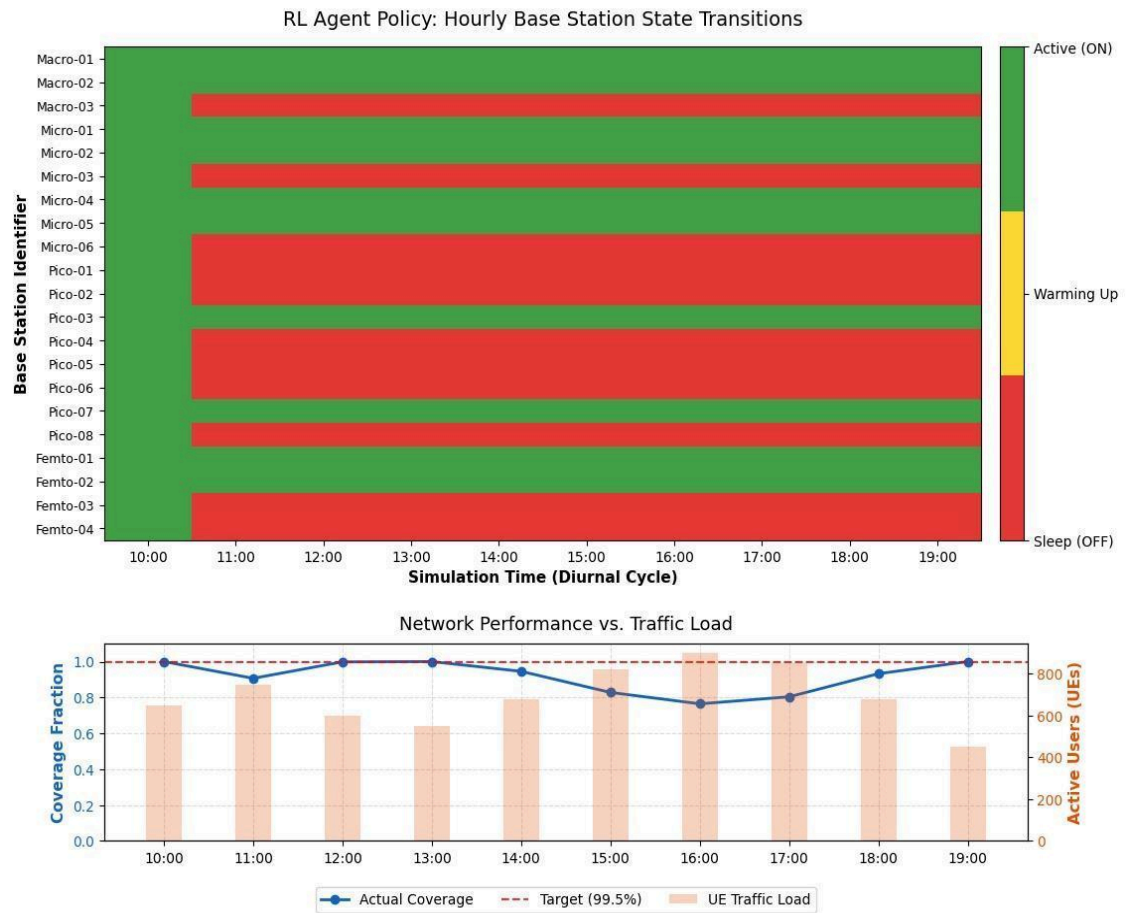


Figure 9.3: Diurnal Network Performance and Traffic Load

Figure 9.3 shows the agent's behaviour across a simulated day. It puts several small cells (Pico/Femto) and one larger cell (Macro/Micro) into persistent sleep to save power. However, around peak load (16:00–17:00, with ~820–900 UEs), coverage drops below the target. This confirms the earlier observation that I mentioned about the current policy being too aggressive with energy savings and disrespecting network coverage during the busiest hour.

9.6. Policy Evaluation and Benchmarking

I benchmarked the trained RL-PPO policy against four baseline heuristics. Each baseline represents a different standard approach to network management:

1. **Always-On:** Here, every BS is permanently forced ON, thereby drawing maximum power at all times. Obviously it guarantees the best possible coverage but acts as the worst-case scenario for energy efficiency.
2. **Random:** It is an experimental mathematical baseline, where I take a uniformly random binary decision (ON or OFF) for every BS at each time step, provided the hardcoded rule is that the main Macro cell 1 is never switched off, preventing the network from completely crashing. Just an inspiration tweaked from [15] to never turn off an macro
3. **Threshold:** A reactive approach and a load based heuristic commonly used in traditional networks. If a Pico or Femto cell's load drops below 10% for two consecutive time steps, the policy turns it off. If the load spikes, the counter resets. The macro still remains active.
4. **Greedy:** This policy uses the same TRUS introduced earlier in G-SCP. At each time step, it forces the Macro cell ON, sorts the remaining base stations by their TRUS and sequentially turns ON the most trusted cells until an estimated coverage target is reached.

The evaluation suite was run for 3 episodes, which corresponds to 36 environment steps (1 episode = 12 steps, representing 12 hours from 10:00 to 22:00). But note that in 1-hour step the agent takes, the environment runs 60 underlying physical micro-steps to simulate user movement, SINR changes, and handover.

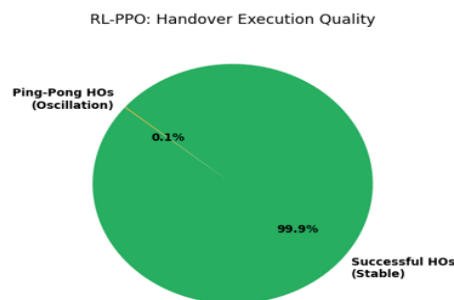


Figure 9.4: RL-PPO handover execution quality (99.9% successful, 0.1% ping-pong)

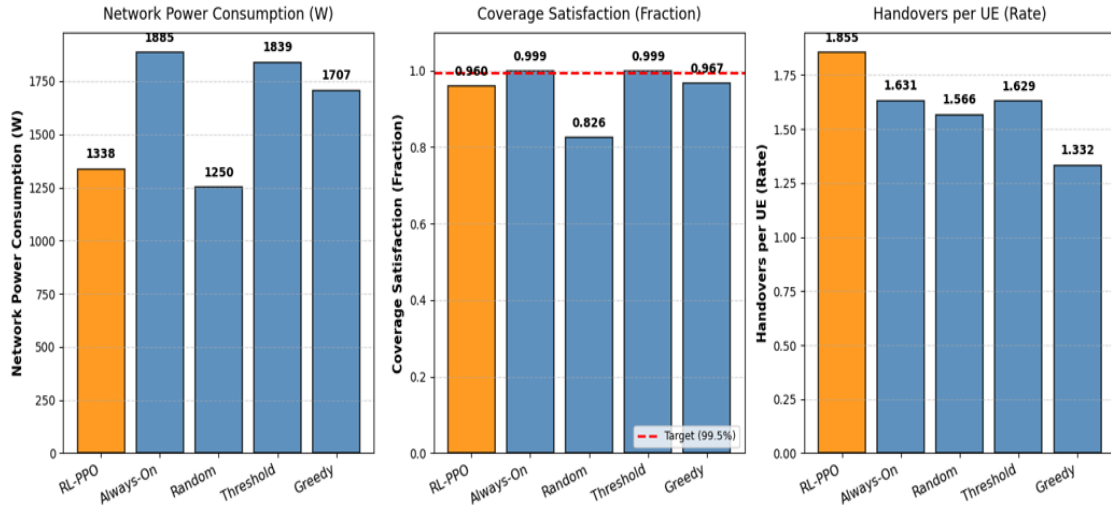


Figure 9.5: Comprehensive multi-objective policy benchmarking vs baseline heuristics: network power, coverage satisfaction (with 99.5% target line) and handovers per UE

The RL agent learnt an aggressive energy-saving strategy, consuming only 1338 W. This yields a power saving of roughly 29.0% compared to Always-On (1885 W) and 21.6% compared to Greedy (1707 W).

However, this efficiency comes at a cost. The RL-PPO policy trades off coverage for energy, missing the strict 0.995 target by settling at 0.960. It also triggers the highest handover rate among the methods (1.855 handovers per UE). On a positive note, despite the high frequency, the handover executions are stable and successful, with only 0.1% resulting in unwanted ping-pong oscillations.

9.7. Handover Evaluation and Spatial Analysis

To understand the real-world impact of the agent's decisions, I ran a 60 second micro-simulation during the busiest hour (20:00–21:00) to differentiate the RL-PPO policy against the Always-On baseline.

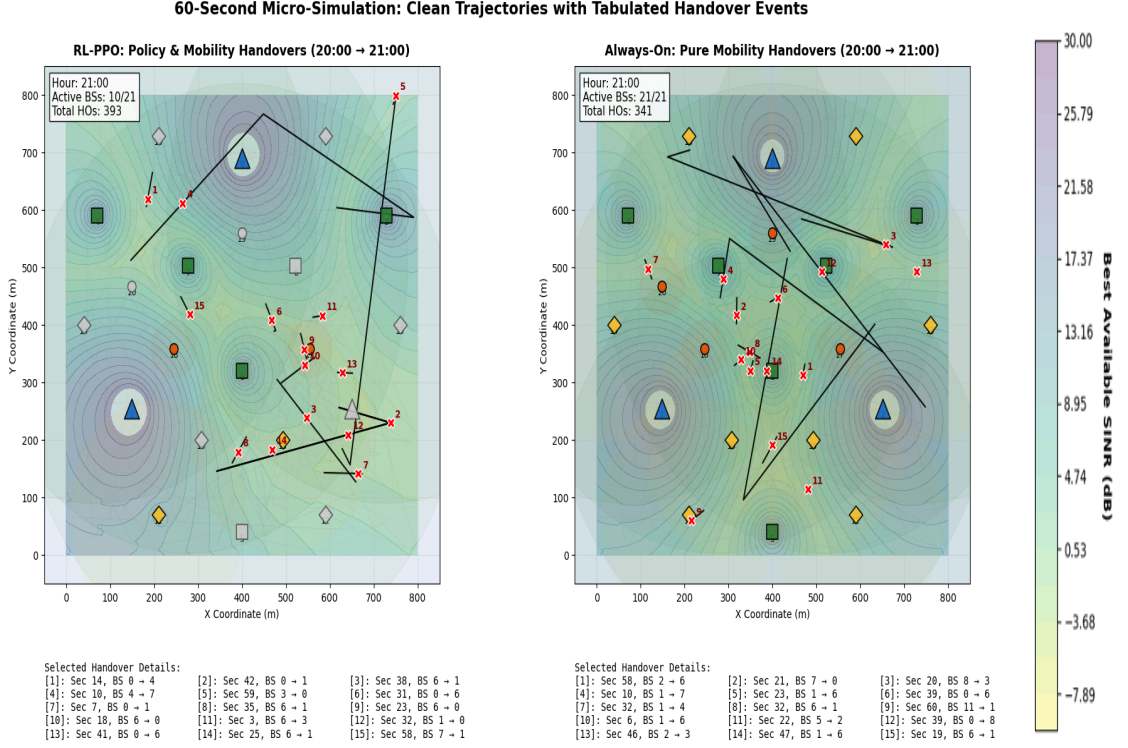


Figure 9.6: Comparison of user trajectories and handover events between the trained RL-PPO policy (left) and the Always-On baseline (right) during the 60 second micro simulation from 21:00 to 22:00.

The black lines show the paths (trajectories) that specific users traveled during that 60-second window. The red crosses indicate the exact second and location along those paths where the user's connection switched from one BS to the next

A core trade-off is easily observed from Figure 9.5 The RL-PPO agent aggressively saves power by keeping only 10 out of the 21 base stations active. However, this forces users to associate with a sparser network of cells. As users move, this wider spacing causes more frequent cell boundary crossings, resulting in 393 total handovers. The Always-On baseline keeps all 21 base stations active. Because the network is denser, moving users cross boundaries less frequently, resulting in only 341 pure mobility handovers.

Also, to visualise how the RL agent manages the network topology dynamically, I plotted the spatial layout at three key points during the diurnal cycle: morning (10:00), peak load (15:00), and evening (19:00).

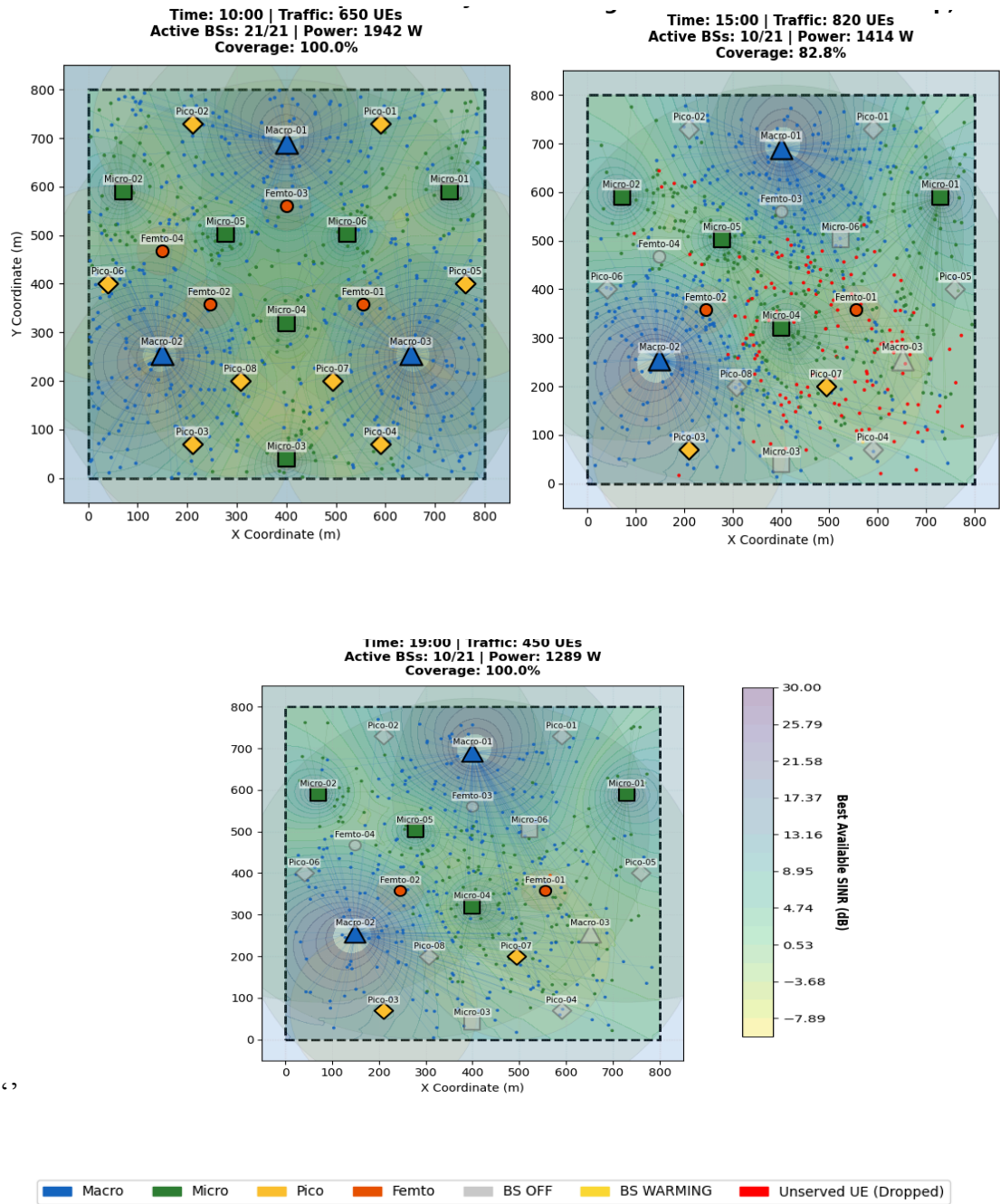


Figure 9.7: Spatial Analysis of RL Agent Decisions Across the Diurnal Cycle for SINR, Load and Topology. Blue and green areas indicate strong signal, while yellow and transparent areas indicate poor coverage. The small dots represent active users (UEs), with red dots specifically highlighting users who have been dropped due to lack of coverage.

- **10:00 (Low Traffic, 650 UEs):** At the start of the simulation, the agent keeps all 21 base stations fully active. The network draws a high 1942 W of power, but it easily achieves 100% coverage. Note that the users are safely connected across the dense grid.
- **15:00 (Peak Traffic, 820 UEs):** During the peak hour, the agent aggressively pivots to save power. It shuts down 11 base stations relying on only 10 active cells. While this drops power consumption significantly to 1414 W, the sparse topology could not handle the high user density. We can clearly see large pockets of poor SINR and numerous red dots, indicating that coverage has crashed to an unacceptable 82.8%.
- **19:00 (Medium Traffic, 450 UEs):** As traffic eases in the evening, the agent maintains the same sparse 10-cell configuration. However, because there are far fewer active users competing for bandwidth, the remaining active cells are sufficient to cover the load. The network achieves 100% coverage while drawing only 1289 W.

In short, the spatial sequence illustrates the current flaw of agent's behaviour, which has learned a rigid, pseudo-effective 'sleep' configuration that works for low-to-medium traffic, but it fails to wake those cells back up when the traffic spikes at peak hours.

10. Limitations and Future Work

While this thesis successfully demonstrates the viability of learned base-station activation, several practical and structural limitations remain, each pointing toward clear directions for future research.

10.1. Optimality and Decision Time in Graph-SCP

The semi-supervised Graph-SCP method is an approximation, not an exact optimum. The power gap here is around 40% compared to the ILP. Occasionally, the coverage also dips to sub-optimal levels under heavy load. If we consider an end-to-end pipeline from instantiating the network instance as a graph representation to making a prediction via GSCP, the average time is easily >1 second and does not yet meet the strict 1 ms decision-time target. This is owed to Python-level graph construction and heuristic repair rather than the neural forward pass.

To improve performance under heavy traffic, future models could use Graph-SCP for a fast initial guess, and only use the slower, exact ILP solver to "repair" the hardest cases. To hit the 1 ms speed target, the neural network needs to be compiled (using tools like TorchScript) and separated from the slower Python code. We can also save time by precomputing the network graph and only updating the parts that change between time steps.

10.2. Reinforcement Learning and Coverage Safety

The semi-supervised Graph-SCP method is an approximation, not an exact optimum. The RL controller successfully learns to save energy, but it misses the strict 99.5% coverage target (reaching only 96.0%). Because the agent turns off cells too aggressively, it forces higher-than-normal handovers. Also, the model hasn't fully finished learning and not converged from my end.

The reward system needs to strictly enforce the coverage rule and limit how often handovers can happen. I could think of a safety layer that prevents the agent from turning off essential cells. Also, instead of training the RL agent from scratch, we could give it a head start by using the static decisions already made by Graph-SCP to guide its early learning.

10.3. Simulation Fidelity and Channel Modelling

The simulator relies on SINR-threshold coverage rather than a full Shannon-rate formulation. MIMO is modelled simply via link-budget beamforming gain rather than full spatial multiplexing, and the power model uses standard theoretical constants rather than hardware-specific, measured curves.

I should definitely research how to integrate higher-fidelity physical layers, such as full 3GPP TR 38.901 spatial channel models and explicit multi-user MIMO capabilities. Calibrating the power and propagation models against real-world hardware measurements and testing robustness against non-synthetic traffic forecasts will offer a new perspective on industrial deployment.

Summary

In this thesis, I addressed the problem of energy-efficient base station activation in a dense four-tier heterogeneous network (HetNet), with the goal of minimizing total network power consumption while maintaining reliable user coverage across dynamically varying traffic conditions.

First, the network model was presented, covering the four-tier topology of macro, micro, pico, and femto base stations, the multi-band radio propagation model, load-dependent power consumption based on the EARTH framework, and a diurnal traffic model reflecting realistic off-peak and peak demand patterns. The activation problem was then formulated as a capacitated, power-weighted Set Cover Problem and solved exactly using an Integer Linear Program (ILP), which served both as a performance oracle and as the source of training labels for the learning-based methods.

To enable fast, practical decision-making, a semi-supervised GraphSAGE model was developed by adapting the Graph-SCP framework to the wireless base station activation setting. A novel contribution of this work is the Topology-Resource Utility Score (TRUS), a pre-association node feature that fuses channel coverage quality with a geometry-based load estimate, avoiding circular dependency on the very association decisions the model is designed to predict. Ablation studies confirmed that injecting TRUS at the node level was the primary driver of coverage quality, and that edge-only injection was insufficient. When evaluated, the adapted Graph-SCP recovered about only half of the ILP’s optimal energy savings, but saving roughly 30% power relative to the greedy baseline at high mean coverage, and it ran at almost eight times the speed of the ILP solver. The Graph-SCP model also generalised across traffic loads beyond its training range.

Finally, reinforcement learning was implemented using a proximal policy optimization controller in a mobility-aware environment with explicit A3 handover mechanics. The agent demonstrated that the dynamic problem is learnable and achieved meaningful energy savings versus an always-on baseline, but it did not yet satisfy the strict coverage target. The policy was only partially stabilised. Cell-level handover execution is reliable, but the RL policy achieves this at the cost of elevated handover rates. Given the relatively small training budget, I treat this RL component as an exploratory prototype rather than a fully developed controller.

References

- [1] J. Lorincz, T. Garma, G. Petrovic, "Measurements and Modelling of Base Station Power Consumption under Real Traffic Loads," *Sensors*, vol. 12, no. 4, 2012. DOI: 10.3390/s120404281.
<https://pmc.ncbi.nlm.nih.gov/articles/PMC3355411/>
- [2] A. Fayad and T. Cinkler, "Energy-Efficient Joint User and Power Allocation in 5G Millimeter Wave Networks: A Genetic Algorithm-Based Approach," *IEEE Access*, 2024. DOI: 10.1109/ACCESS.2024.10418932.
<https://ieeexplore.ieee.org/document/10418932>
- [3] W. L. Hamilton, Z. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," *NeurIPS*, 2017. arXiv:1706.02216.
<https://arxiv.org/abs/1706.02216>
- [4] S. Mitchell, M. O'Sullivan, and I. Dunning, "PuLP: A Linear Programming Toolkit for Python," 2011.
<https://coin-or.github.io/pulp/>
- [5] N. Bhushan, J. Li, D. Malladi, R. Gilmore, D. Brenner, A. Damnjanovic, R. T. Sukhavasi, C. Patel, and S. Geirhofer, "Network Densification: The Dominant Theme for Wireless Evolution into 5G," *IEEE Communications Magazine*, vol. 52, no. 2, pp. 82–89, Feb. 2014.
<https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=&arnumber=6736747&tag=1>
- [6] G. Auer, V. Giannini, C. Desset et al., "How Much Energy Is Needed to Run a Wireless Network?," *EARTH Project Deliverable D2.3*, *IEEE Wireless Communications*, vol. 18, no. 5, pp. 40-49, 2011.
<https://ieeexplore.ieee.org/document/6056691>
- [7] 3GPP TR 38.901 V17.0.0, "Study on Channel Model for Frequencies from 0.5 to 100 GHz," 2022.
https://www.etsi.org/deliver/etsi_tr/138900_138999/138901/17.00.00_60/tr_138901v170000p.pdf
- [8] H. Holtkamp, G. Auer, V. Giannini, and H. Haas, "A Parameterized Base Station Power Model," *IEEE Communications Letters*, vol. 17, no. 11, pp. 2033–2035, 2013. DOI: 10.1109/LCOMM.2013.091213.131042
<https://arxiv.org/abs/1411.1571>
- [9] A. Bousia, E. Kartsakli, L. Alonso, and C. Verikoukis, "Intelligent Base Station Management in Greener Traffic-Aware Cellular Networks," in *Proc. IEEE PIMRC*, 2014. DOI: 10.1109/PIMRC.2014.6929242.
<https://ieeexplore.ieee.org/document/6929242>
- [10] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017. DOI: 10.48550/arXiv.1707.06347.

<https://arxiv.org/abs/1707.06347>

- [11] T. Cai, S. Luo, K. Xu, D. He, T.-Y. Liu, and L. Wang, "GraphNorm: A Principled Approach to Accelerating Graph Neural Network Training," in Proc. ICML, 2021. arXiv:2009.03294.
<https://arxiv.org/abs/2009.03294>
- [12] Z. Shafi, B. A. Miller, T. Eliassi-Rad, and R. S. Caceres, "Graph-SCP: Accelerating Set Cover Problems with Graph Neural Networks," arXiv:2310.07979, 2023.
<https://arxiv.org/abs/2310.07979>
- [13] 3GPP TS 38.214 V17.x, "NR; Physical layer procedures for data," 2022. https://www.3gpp.org/ftp/Specs/archive/38_series/38.214/
- [14] A. Goldsmith, Wireless Communications. Cambridge University Press, 2005. ISBN: 9780521837163.
- [15] K. Tan, D. Bremner, J. Le Kernec, Y. Sambo, L. Zhang, and M. A. Imran, "Graph Neural Network-Based Cell Switching for Energy Optimization in Ultra-Dense Heterogeneous Networks," Scientific Reports, vol. 12, p. 21581, 2022. DOI: 10.1038/s41598-022-25800-3.
<https://doi.org/10.1038/s41598-022-25800-3>
- [16] E. Rossi, B. Chamberlain, F. Frasca, D. Eynard, F. Monti, and M. M. Bronstein, "Temporal Graph Networks for Deep Learning on Dynamic Graphs," ICML Workshop, 2020. arXiv:2006.10637.
<https://arxiv.org/abs/2006.10637>
- [17] A3 Handover and Time-To-Trigger parameters: M. Yassin, M. El Hassan, and R. Haddad, "A3 Event Handover Parameter Optimization," Wireless Communications and Mobile Computing, 2022. DOI: 10.1155/2022/2266282.
<https://downloads.hindawi.com/journals/wcmc/2022/2266282.pdf>
- [18] M. Hoffmann, A. Kliks, P. Kryszkiewicz, and G. Koudouridis, "A Reinforcement Learning Approach for Base Station On/Off Switching in Heterogeneous M-MIMO Networks," in Proc. IEEE WoWMoM, 2020. DOI: 10.1109/WoWMoM49955.2020.00038.
<https://ieeexplore.ieee.org/document/9131389>
- [19] T. Rumeng, W. Tong, S. Ying, and H. Yanpu, "Intelligent Energy Saving Solution of 5G Base Station Based on Artificial Intelligence Technologies," in Proc. IEEE EMC+SIPI, 2021. DOI: 10.1109/EMC/SI/PI/EMCEurope52599.2021.9559261.
- [20] Q. Cappart, D. Chételat, E. Khalil, A. Lodi, C. Morris, and P. Veličković, "Combinatorial Optimization and Reasoning with Graph Neural Networks," JMLR, vol. 24, 2023. arXiv:2102.09544.
<https://arxiv.org/abs/2102.09544>
- [21] J. Mirzaei, J. Mitra, and G. Poitau, "Graph Attention Network for Optimal User Association in Wireless Networks," arXiv:2505.16347, 2025.
<https://arxiv.org/abs/2505.16347>

- [22] A. Fayad, T. Cinkler, and J. Rak, "5G Millimeter Wave Network Optimization: Dual Connectivity and Power Allocation Strategy," IEEE Access, 2023.
<https://ieeexplore.ieee.org/document/10198445>
- [23] Z. Shafi et al., "A Comprehensive Survey of LP/ILP Approaches for Resource Allocation in 5G and Beyond," arXiv:2502.15585, 2025.
<https://arxiv.org/abs/2502.15585>

Appendix

A. Algorithmic Implementations

A.1. Greedy Capacitated Set Cover Heuristic Algorithm

```
1: Input: Feasible sets  $\{S_j\}$ , costs  $\{c_j\}$ , capacities  $\{C_j\}$ , target coverage  $\tau$ 
2: Initialize active set  $A \leftarrow \emptyset$ , covered users  $\leftarrow \emptyset$ , assignment  $\leftarrow \emptyset$ 
3: while  $|\text{covered}| < \tau \cdot |U|$  and  $\exists$  feasible candidate do
4:   for each inactive BS  $j$  with spare capacity do
5:     // Count new users the BS brings (subtracting already covered)
6:      $\text{gain}_j \leftarrow |S_j \setminus \text{covered}|$ 
7:     // Calculate the energy cost to wake the BS
8:      $\text{delta}_j \leftarrow \text{incremental activation power}$ 
9:     // Compute cost-effectiveness ratio (Users per Watt)
10:     $\text{score}_j \leftarrow \text{gain}_j / \max(\text{delta}_j, \epsilon)$ 
11:   end for
12:   // Select the BS with the absolute highest score
13:    $j^* \leftarrow \text{argmax}(\text{score}_j)$ 
14:   // If the best BS provides 0 new users, all reachable UEs are connected
15:   if  $\text{gain}_{\{j^*\}} == 0$  then
16:     break
17:   end if
18:   // Assign the winning BS to the active set
19:    $A \leftarrow A \cup \{j^*\}$ 
20:   // Strictly maintain the capacity constraint per BS
21:   assign up to  $C_{\{j^*\}}$  uncovered UEs from  $S_{\{j^*\}}$  to  $j^*$ 
22:   update covered users
23: end while
24: return  $A$ , assignment
```

A.2. TRUS Feature Integration

```
1: Input: UE positions, BS positions and tiers, feasible-link matrix F, capacities C
2: Initialize arrays  $\phi_{c,j}$ ,  $\hat{n}_j$ ,  $\rho_j$ , and TRUSj for all base stations j
3: for each base station j do // Local node features
4:   for each inactive BS j with spare capacity do
5:     Compute normalized channel utility  $\phi_{c,j}$  over all feasible UE links of j
6:     Estimate provisional demand  $\hat{n}_j$  via tier-weighted Voronoi-style assignment
7:     Compute expected load  $\rho_j \leftarrow \min(1, \hat{n}_j / C_j)$ 
8:     Compute TRUSj  $\leftarrow 0.5\phi_{c,j} + \gamma t(1 - \rho_j)$  // Fuses physics and load
9:   end for
10: Compute connectivity estimate CEj using approximate Fiedler-value drop
11: Compute random-walk-with-restart score RWRj on the bipartite graph
12: for each base station j do // Final feature assembly , // TRUS injected to edge
13:   Assemble node feature vector node_feat[j]
14:   for each feasible UE i with Fij = 1 do
15:     Set edge_attr[i, j]  $\leftarrow$  [normalized_channel_weight(i, j), TRUSj]
16:   end for
17: end for
18: Assign dummy edge attributes [0, 0] to universe-UE edges // Prevent NaNs
19: return node_feat, edge_attr
```

B. Source Code Availability

B.1. Graph-SCP Implementation

Author: Praneshraj Tiruppur Nagarajan Dhyaneswar

Filename: AOMTO9_Thesis_Graph_SCP_Semi_Supervised_0706.ipynb

Description: Contains the HetNet physics simulator, ILP Oracle generation, TRUS mathematical formulation, GraphSAGE architecture, semi-supervised training loop, and evaluation benchmarking.

Link: [🔗 AOMTO9_Thesis_Graph_SCP_Semi_Supervised_0706.ipynb](#)

B.2. Reinforcement Learning (RL-PPO) Implementation

Author: Praneshraj Tiruppur Nagarajan Dhyaneswar

Filename: AOMTO9_Thesis_Reinforcement_Learning_V7.ipynb

Description: Contains the HetNet physics simulator, ILP Oracle generation, TRUS mathematical formulation, GraphSAGE architecture, semi-supervised training loop, and evaluation benchmarking.

Link: [🔗 AOMTO9_Thesis_Reinforcement_Learning_V7.ipynb](#)

C. Hyperparameter Configurations

C.1. Graph-SCP Training Configuration

Parameter	Value
Optimiser	AdamW
Learning Rate	10^{-3}
Weight decay	10^{-4}
Gradient clipping	1.0
Learning Rate Scheduler	CosineAnnealing
Learning rate decay	$\eta_{\min}=10^{-6}$
Max epochs	100
Early stopping patience	15 epochs
Train / Validation Split	85% / 15%
Hidden dimension	128
GNN layers	3
Dropout	0.3
Model parameters	117,505

Table C.1: Training Hyperparameters for G-SCP

C.2. RL-PPO Training Configurations

Parameter	Value
Discount γ	0.99
Hidden layer size	256
Epochs per update	10
Total PPO steps	20,000
Learning Rate	0.0005
Batch Size	125
Advantage	GAE- λ
Surrogate objective	Clipped
Exploration	Entropy Annealing
Trainable parameters	421,654

Table C.2: Training Hyperparameters for RL-PPO

Acknowledgements

First and foremost, I express my deepest gratitude to my supervisor, Dr. Tibor Cinkler, for his invaluable guidance, support, and immense patience throughout my Master's studies.

I also wish to thank Dr. Abdulhalim Fayad. His technical advice, parameter suggestions, and analytical feedback were instrumental in successfully shaping the network model.

Finally, my most profound thanks go to my father, my mother, and my sister. Your unwavering support and belief in me were my greatest source of strength, carrying me through the most difficult moments of this journey.

Declaration on the Use of Generative Artificial Intelligence

I have not used any generative AI tools.

✓ **I have used generative AI tools.** I have verified the content generated by AI, ensured the accuracy of the outputs, and properly indicated each instance of use in the table below.

Usage type	Name of Generative AI Tool(s)	Affected Sections (chapter, page number, reference)	Estimated Proportion of Use (per usage type)
Literature Review	Gemini 3.1 Pro; Perplexity (GPT-based assistant)	Introduction , 4-tier HetNet deployment , Path loss , BS Power Consumption Model , Graph-SCP , Ablation Methodologies , MDP , PPO	17%
Brief Summary of the Prompt	Identification of research papers similar in concepts, Deeper formulae explanation of Path loss and power models, types of ablation studies on higher level, conceptual explanations in RL		
Program Code Generation	Gemini 3.1 Pro, Perplexity	Appendix	49%
Brief Summary of the Prompt	I implemented breakdown and logic flow. I needed support to replicate my thoughts into a code. Extensively used in visualising via pyplot		
Generating New Ideas or Solution Proposals	Gemini 3.1 Pro, Perplexity	TRUS	1%
Brief Summary of the Prompt	Prompting the AI to decide the most mathematically sound approach out of my combinations of list of network parameters for BS importance.		
Creating an Outline (text structure, bullet points)	Gemini 3.1 Pro, Perplexity	Table of Content	2%
Brief Summary of the Prompt	Structural ordering for sensible flow of the particular chapters		
Creating Text Blocks	Gemini 3.1 Pro, Perplexity	Thesis	8%

Brief Summary of the Prompt	Language polishing, transitions, and reformulation of explanations.		
Generating Images for Illustrative Purposes	Gemini 3.1 Pro, Perplexity	Motivation , Graph-SCP Pipeline	3%
Brief Summary of the Prompt	Generate Latex TikZ code for diagram used in a research paper, and tweak it pertaining to my G-SCP flow and attributes		
Data Visualization, Generating Charts Based on Data Points	Gemini 3.1 Pro, Perplexity	Dataset Composition , GSCP Training and Convergence ,	5%
Brief Summary of the Prompt	I already saved the attributes of the evaluation dataset for GSCP in a csv. Only 2 plots I missed in the code and later generated it due to significance		
Preparing a Presentation	-	-	-
Brief Summary of the Prompt	-		
Other (please specify)	Gemini 3.1 Pro, Perplexity	Thesis	5%
Brief Summary of the Prompt	Consistency checking and Examiner Style Critique		
Aggregated Percentage Value (for the core part of the task)			29%
Brief Textual Justification of the Aggregated Value: Highest usage occurred in programming syntax. AI was strictly utilized to accelerate coding execution and assess the correctness in mathematical formulation of the TRUS metric, MDP state-space design, the architecture framework of Graph-SCP model, and critical analysis of model results and reimproving it			